

Business Case: Walmart - Confidence Interval and CLT

(Made by: Harshavardhan Patra Data Analyst)



About Walmart

Walmart is an American multinational retail corporation that operates a chain of supercenters, discount departmental stores, and grocery stores from the United States. Walmart has more than 100 million customers worldwide.

Business Problem

The Management team at Walmart Inc. wants to analyze the customer purchase behavior (specifically, purchase amount) against the customer’s gender and the various other factors to help the business make better decisions. They want to understand if the spending habits differ between male and female customers: Do women spend more on Black Friday than men? (Assume 50 million customers are male and 50 million are female).

Dataset

The company collected the transactional data of customers who purchased products from the Walmart Stores during Black Friday. The dataset has the following features: Dataset link: [Walmart_data.csv](#)

User_ID:	User ID
Product_ID:	Product ID
Gender:	Sex of User
Age:	Age in bins
Occupation:	Occupation(Masked)
City_Category:	Category of the City (A,B,C)
StayInCurrentCityYears:	Number of years stay in current city
Marital_Status:	Marital Status
ProductCategory:	Product Category (Masked)
Purchase:	Purchase Amount

```
In [66]: #importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import t
import warnings
warnings.filterwarnings('ignore')
import copy
```

```
In [67]: walmart_dataset = pd.read_csv("/content/drive/MyDrive/Walmart case study /walmart_data.txt")
```

```
In [68]: df = walmart_dataset.copy()
```

```
In [69]: df.head()
```

Out[69]:	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category	Purchase
0	1000001	P00069042	F	0-17	10	A	2	0	3	8370
1	1000001	P00248942	F	0-17	10	A	2	0	1	15200
2	1000001	P00087842	F	0-17	10	A	2	0	12	1422
3	1000001	P00085442	F	0-17	10	A	2	0	12	1057
4	1000002	P00285442	M	55+	16	C	4+	0	8	7969

Defining Problem Statement and Analyzing basic metrics (10 Points)

Observations on shape of data, data types of all the attributes, conversion of categorical attributes to 'category' (If required), statistical summary

In [70]:

df.shape

Out[70]: (550068, 10)

In [71]:

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User_ID               550068 non-null  int64
1   Product_ID           550068 non-null  object
2   Gender               550068 non-null  object
3   Age                  550068 non-null  object
4   Occupation           550068 non-null  int64
5   City_Category        550068 non-null  object
6   Stay_In_Current_City_Years  550068 non-null  object
7   Marital_Status       550068 non-null  int64
8   Product_Category     550068 non-null  int64
9   Purchase             550068 non-null  int64
dtypes: int64(5), object(5)
memory usage: 42.0+ MB
```

 Insights

- From the above analysis, it is clear that, data has total of 10 features with lots of mixed alpha numeric data.
- Apart from Purchase Column, all the other data types are of categorical type. We will change the datatypes of all such columns to category

In [72]:

df.isna().sum()

Out[72]:

	0
User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category	0
Purchase	0

dtype: int64

 Insights:

No null values detected.

In [73]:

```
for i in df.columns[:-1]:
    df[i] = df[i].astype('category')

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 550068 entries, 0 to 550067
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   User_ID               550068 non-null  category
1   Product_ID           550068 non-null  category
2   Gender               550068 non-null  category
3   Age                  550068 non-null  category
4   Occupation           550068 non-null  category
5   City_Category        550068 non-null  category
6   Stay_In_Current_City_Years  550068 non-null  category
7   Marital_Status       550068 non-null  category
8   Product_Category     550068 non-null  category
9   Purchase             550068 non-null  int64
dtypes: category(9), int64(1)
memory usage: 10.3 MB
```

In [74]:

df.describe(include = 'category')

Out[74]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years	Marital_Status	Product_Category
count	550068	550068	550068	550068	550068	550068	550068	550068	550068
unique	5891	3631	2	7	21	3	5	2	20
top	1001680	P00265242	M	26-35	4	B	1	0	5
freq	1026	1880	414259	219587	72308	231173	193821	324731	150933

 Insights

- User_ID - Among 5,50,068 transactions there are 5891 unique user_id, indicating same customers buying multiple products.
- Product_ID - Among 5,50,068 transactions there are 3631 unique products,with the product having the code P00265242 being the highest seller, with a maximum of 1,880 units sold.
- Gender - Out of 5,50,068 transactions, 4,14,259 (nearly 75%) were done by male gender indicating a significant disparity in purchase behavior between males and females during the Black Friday event.
- Age - We have 7 unique age groups in the dataset. 26 - 35 Age group has maximum of 2,19,587 transactions. We will analyse this feature in detail in future
- Stay_In_Current_City_Years - Customers with 1 year of stay in current city accounted to maximum of 1,93,821 transactions among all the other customers with (0,2,3,4+) years of stay in current city
- Marital_Status - 59% of the total transactions were done by Unmarried Customers and 41% by Married Customers.

Satistical summary of numerical data type columns

In [75]:

```
df.describe()
```

Out[75]:

	Purchase
count	550068.000000
mean	9263.968713
std	5023.065394
min	12.000000
25%	5823.000000
50%	8047.000000
75%	12054.000000
max	23961.000000

 Insights

The purchase amounts vary widely, with the minimum recorded purchase being 12 and the maximum reaching 23961. The median purchase amount of 8047 is notably lower than the mean purchase amount of \$9264, indicating a right-skewed distribution where a few high-value purchases pull up the mean

Duplicate Detection

In [76]:

```
df.duplicated().value_counts()
```

Out[76]:

	count
False	550068

dtype: int64

 Insights

There are no duplicate entries in the dataset

Sanity Check for columns

In [77]:

```
for i in df.columns:
    print('Unique Values in',i,'column are :-')
    print(df[i].unique())
    print('-'*70)
```

```
Unique Values in User_ID column are :-
[1000001, 1000002, 1000003, 1000004, 1000005, ..., 1004588, 1004871, 1004113, 1005391, 1001529]
Length: 5891
Categories (5891, int64): [1000001, 1000002, 1000003, 1000004, ..., 1006037, 1006038, 1006039, 1006040]
-----

Unique Values in Product_ID column are :-
['P00069042', 'P00248942', 'P00087842', 'P00085442', 'P00285442', ..., 'P00375436', 'P00372445', 'P00370293', 'P00371644', 'P00370853']
Length: 3631
Categories (3631, object): ['P00000142', 'P00000242', 'P00000342', 'P00000442', ..., 'P0099642', 'P0099742', 'P0099842', 'P0099942']
-----

Unique Values in Gender column are :-
['F', 'M']
Categories (2, object): ['F', 'M']
-----

Unique Values in Age column are :-
['0-17', '55+', '26-35', '46-50', '51-55', '36-45', '18-25']
Categories (7, object): ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
-----

Unique Values in Occupation column are :-
[10, 16, 15, 7, 20, ..., 18, 5, 14, 13, 6]
Length: 21
Categories (21, int64): [0, 1, 2, 3, ..., 17, 18, 19, 20]
-----

Unique Values in City_Category column are :-
['A', 'C', 'B']
Categories (3, object): ['A', 'B', 'C']
-----

Unique Values in Stay_In_Current_City_Years column are :-
['2', '4+', '3', '1', '0']
Categories (5, object): ['0', '1', '2', '3', '4+']
-----

Unique Values in Marital_Status column are :-
[0, 1]
Categories (2, int64): [0, 1]
-----

Unique Values in Product_Category column are :-
[3, 1, 12, 8, 5, ..., 10, 17, 9, 20, 19]
Length: 20
Categories (20, int64): [1, 2, 3, 4, ..., 17, 18, 19, 20]
-----

Unique Values in Purchase column are :-
[ 8370 15200  1422 ...   135   123   613]
```

 Insights

- The dataset does not contain any abnormal values.
- We will convert the 0,1 in Marital Status column as married and unmarried

```
In [78]: #replacing the values in marital_status column

df['Marital_Status'] = df['Marital_Status'].replace({0:'Unmarried',1:'Married'})
df['Marital_Status'].unique()

Out[78]: ['Unmarried', 'Married']
Categories (2, object): ['Unmarried', 'Married']
```

Visual Analysis - Univariate & Bivariate

- For continuous variable(s): Distplot, countplot, histogram for univariate analysis
- For categorical variable(s): Boxplot
- For correlation: Heatmaps, Pairplots

3.Univariate Analysis

3.1 Numerical Variables

💰 3.1.1 Purchase Amount Distribution

```
In [79]: purchase = df['Purchase']

# Color palette (clean + modern)
hist_color = '#4A90E2'      # blue
box_color = '#50C878'       # emerald green
median_color = '#FF4C4C'    # red
outlier_color = '#2E2E2E'   # dark grey

# Create figure
fig = plt.figure(figsize=(15, 10))
gs = fig.add_gridspec(2, 1, height_ratios=[0.65, 0.35])

# ----- Histogram -----
ax0 = fig.add_subplot(gs[0])

ax0.hist(
    purchase,
```

```

        bins=20,
        color=hist_color,
        edgecolor='black',
        alpha=0.85
    )

ax0.set_xlabel('Purchase Amount', fontsize=12, fontweight='bold')
ax0.set_ylabel('Frequency', fontsize=12, fontweight='bold')
ax0.set_title('Purchase Amount Distribution',
              fontdict={'family': 'serif', 'size': 15, 'weight': 'bold'})

# Remove spines
for spine in ['top', 'left', 'right']:
    ax0.spines[spine].set_visible(False)

# ----- Boxplot -----
ax1 = fig.add_subplot(gs[1])

boxplot = ax1.boxplot(
    purchase,
    vert=False,
    patch_artist=True,
    widths=0.5
)

# Style boxplot
boxplot['boxes'][0].set(facecolor=box_color)
boxplot['medians'][0].set(color=median_color, linewidth=2)

for flier in boxplot['fliers']:
    flier.set(marker='o', markersize=7, markerfacecolor=outlier_color)

for spine in ['top', 'left', 'right']:
    ax1.spines[spine].set_visible(False)

# Five-number summary
q1 = purchase.quantile(0.25)
median = purchase.median()
q3 = purchase.quantile(0.75)

lower_whisker = boxplot['whiskers'][0].get_xdata()[1]
upper_whisker = boxplot['whiskers'][1].get_xdata()[1]

summary_points = [lower_whisker, q1, median, q3, upper_whisker]

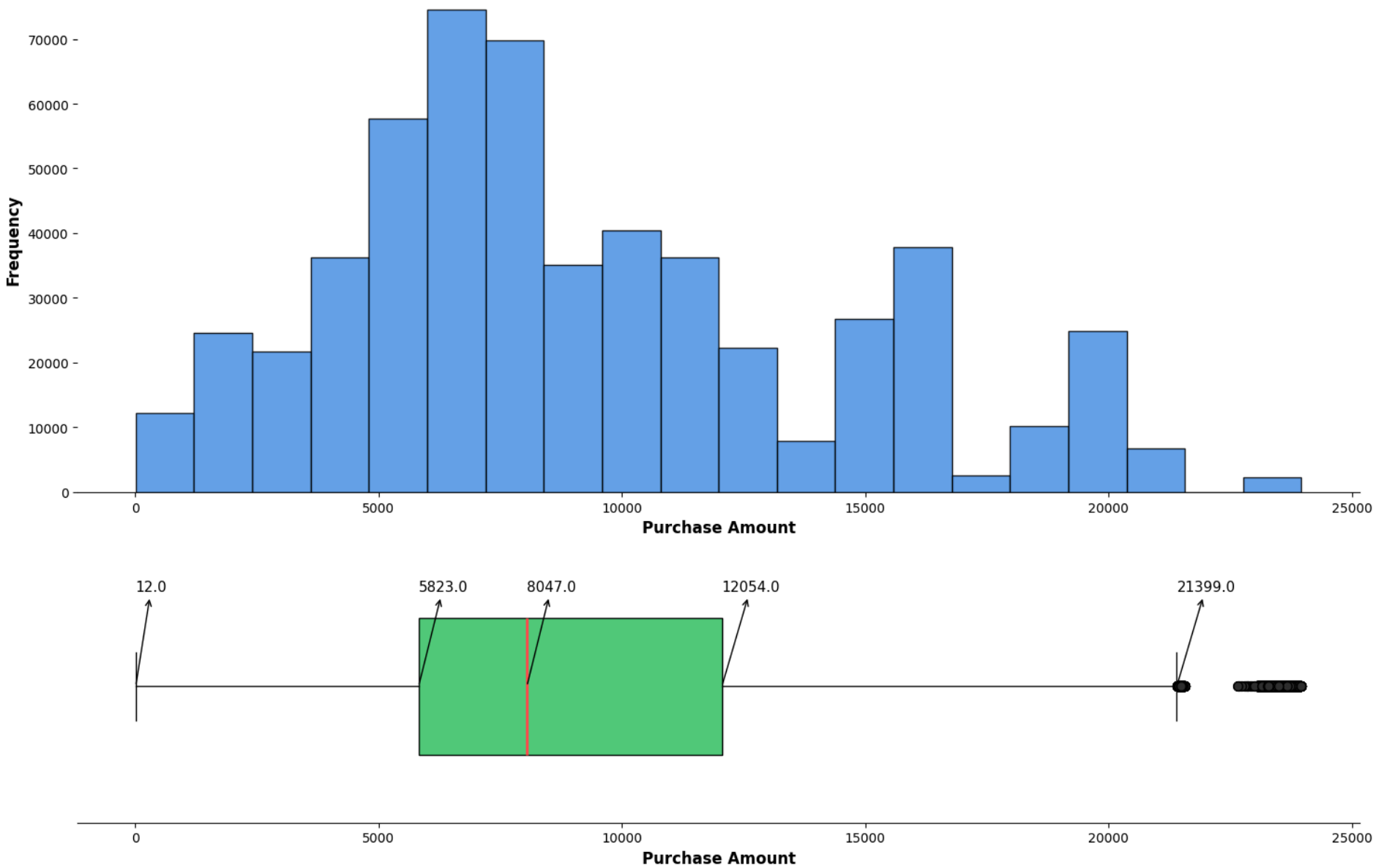
# Annotate
for x in summary_points:
    ax1.annotate(
        text=f"{x:.1f}",
        xy=(x, 1),
        xytext=(x, 1.35),
        fontsize=11,
        arrowprops=dict(arrowstyle="<-", lw=1)
    )

ax1.set_yticks([])
ax1.set_xlabel('Purchase Amount', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()

```

Purchase Amount Distribution



Calculating the Number of Outliers:

- As seen above, Purchase amount over 21399 is considered as outlier. We will count the number of outliers as below

```
In [80]: len(df.loc[df['Purchase'] > 21399, 'Purchase'])
```

Out[80]: 2677

🔍 Insights

- Outliers
 - There are total of 2677 outliers which is roughly 0.48% of the total data present in purchase amount. We will not remove them as it indicates a broad range of spending behaviors during the sale, highlighting the importance of tailoring marketing strategies to both regular and high-value customers to maximize revenue.
- Distribution
 - Data suggests that the majority of customers spent between 5,823 USD and 12,054 USD, with the median purchase amount being 8,047 USD.
 - The lower limit of 12 USD while the upper limit of 21,399 USD reveal significant variability in customer spending

3.2 Categorical Variables

3.2.1 🧑🏽 Gender, 👫 Marital Status and 🌐 City Category Distribution

```
In [81]: # Create figure
fig = plt.figure(figsize=(15, 6))
gs = fig.add_gridspec(1, 3)

# Common color palette
colors_2 = ['#4A90E2', '#2E2E2E']
colors_3 = ['#4A90E2', '#2E2E2E', '#9BB7C4']

# ----- Function to reduce repetition -----
def plot_bar(ax, series, title, colors):
    counts = series.value_counts()
    percentages = counts / counts.sum() * 100

    bars = ax.barh(counts.index, percentages, color=colors)

    # Add percentage Labels
    for i, v in enumerate(percentages):
        ax.text(v + 1, i, f'{v:.1f}%', va='center', fontsize=11)

    ax.set_title(title, fontdict={'family': 'serif', 'size': 15, 'weight': 'bold'})
    ax.set_xlim(0, 100)

# Clean Look
for spine in ['top', 'right', 'left']:
    ax.spines[spine].set_visible(False)
```



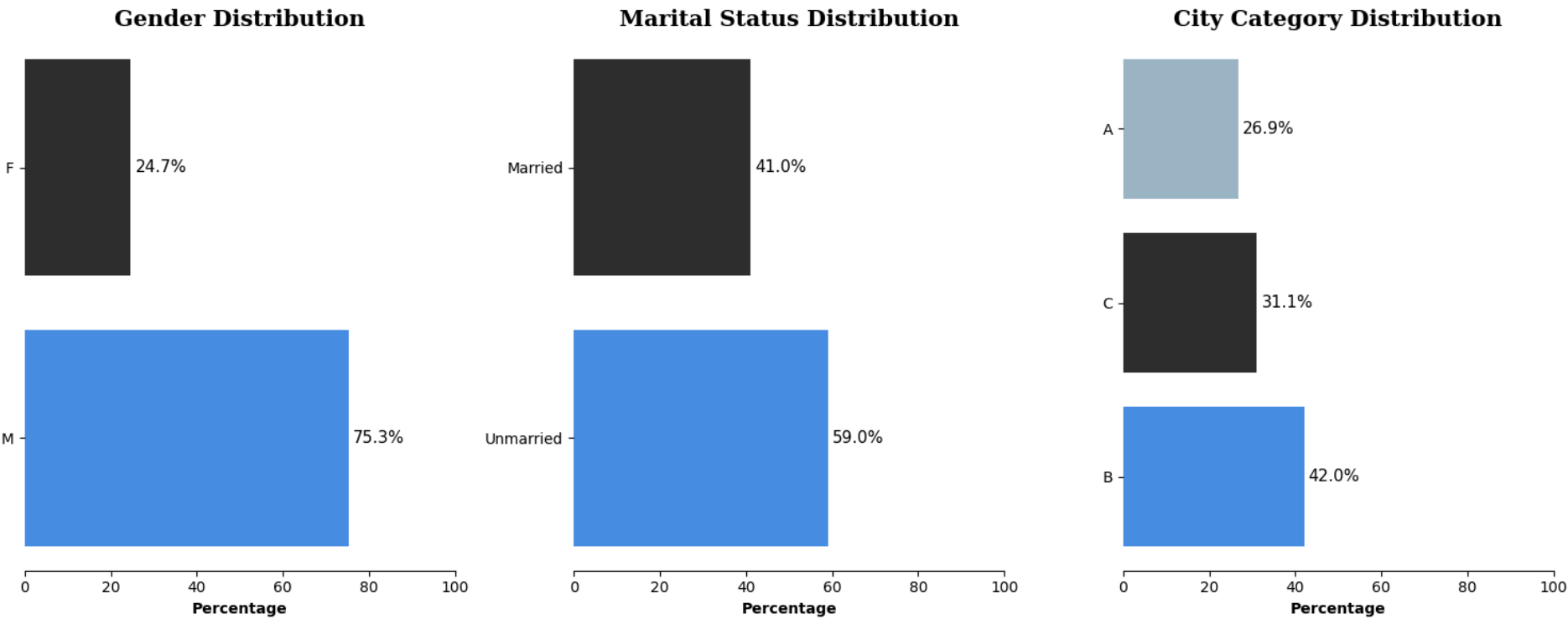
```
ax.set_xlabel('Percentage', fontweight='bold')

# ----- Gender -----
ax0 = fig.add_subplot(gs[0, 0])
plot_bar(ax0, df['Gender'], 'Gender Distribution', colors_2)

# ----- Marital Status -----
ax1 = fig.add_subplot(gs[0, 1])
plot_bar(ax1, df['Marital_Status'], 'Marital Status Distribution', colors_2)

# ----- City Category -----
ax2 = fig.add_subplot(gs[0, 2])
plot_bar(ax2, df['City_Category'], 'City Category Distribution', colors_3)

plt.tight_layout()
plt.show()
```



🔍 Insights

The dataset is **male-dominated (75.3%)**, indicating that overall customer behavior is largely driven by male shoppers, and insights may be skewed toward this segment.

Unmarried customers (59%) form the majority, suggesting stronger participation from younger or independent consumers, making them a key target group.

City B (42%) leads in transactions, followed by City C and A, highlighting it as the most active market, while City A shows relatively lower engagement and potential for growth.

3.2.2 📅 Customer Age Distribution

```
In [82]: # Order
age_order = ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']

# Colors
color_map = {
    '0-17': '#C7D9DD',
    '18-25': '#A6C8D0',
    '26-35': '#4A90E2',
    '36-45': '#5A5A5A',
    '46-50': '#6BA292',
    '51-55': '#7A86A1',
    '55+': '#8FAF6A'
}

# Data
age_counts = df['Age'].value_counts().reindex(age_order)
age_perc = (age_counts / age_counts.sum() * 100).round(1)

age_counts = age_counts.dropna()
age_perc = age_perc.loc[age_counts.index]

bar_colors = [color_map[age] for age in age_counts.index]

# ----- Figure -----
fig, ax = plt.subplots(figsize=(17, 6))

bars = ax.bar(age_counts.index, age_counts.values,
              color=bar_colors, width=0.7, zorder=3)

# Labels
for bar, val, pct in zip(bars, age_counts.values, age_perc.values):
    ax.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height() + 4000,
        f'{val}\n({pct:.1f}%)',
        ha='center',
```

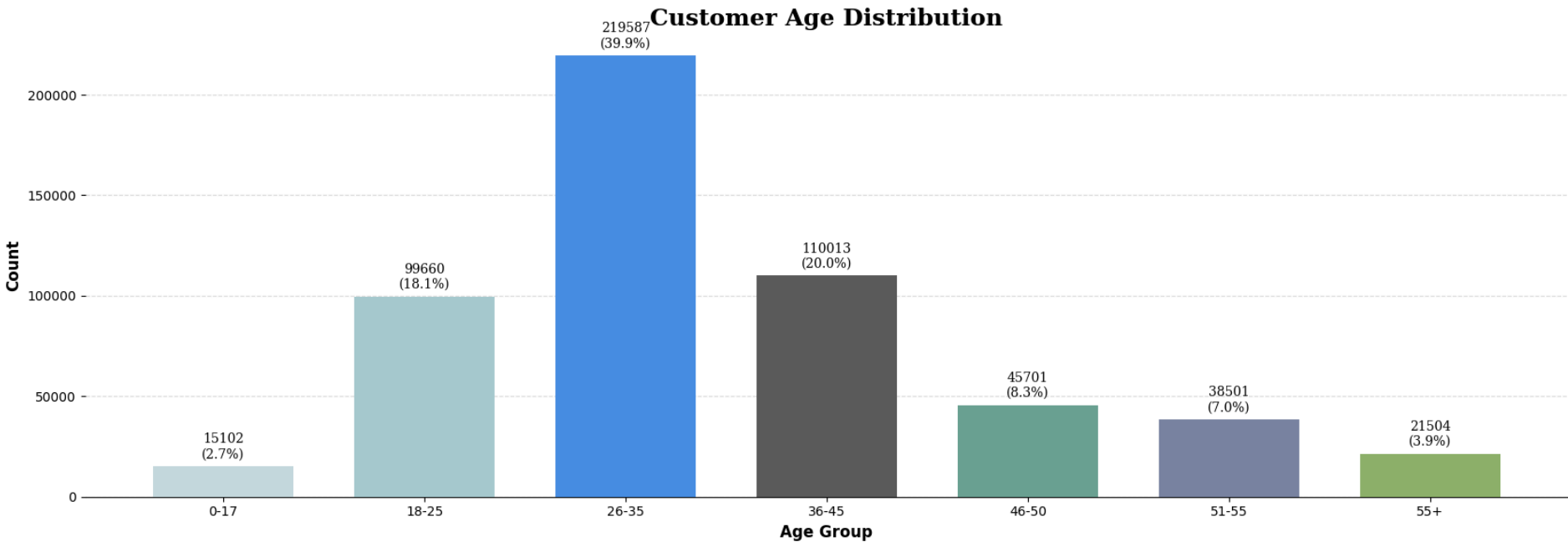
```
        fontsize=10,
        fontfamily='serif'
    )

# Style
ax.grid(axis='y', linestyle='--', linewidth=0.8, alpha=0.4, zorder=0)

for s in ['top', 'right', 'left']:
    ax.spines[s].set_visible(False)

ax.set_ylabel('Count', fontsize=12, fontweight='bold')
ax.set_xlabel('Age Group', fontsize=12, fontweight='bold')
ax.set_title('Customer Age Distribution',
            fontsize=18, fontweight='bold', fontfamily='serif')

plt.tight_layout()
plt.show()
```



🔍 Insights

The **26–35 age group dominates ($\approx 40\%$)**, making it the primary customer segment and key driver of Black Friday activity.

The **18–25 (18%) and 36–45 (20%) groups** also contribute significantly, indicating strong engagement across young and mid-career consumers.

Participation **drops sharply beyond 45**, with each older segment contributing less than 10%, showing lower engagement among older customers.

The **0–17 and 55+ groups ($<5\%$)** have minimal presence, suggesting limited targeting or relevance for these segments during the event.

3.2.3 🏠 Customer Stay In current City Distribution

```
In [83]: # Proper order
stay_order = ['0', '1', '2', '3', '4+']

# Color palette (consistent with previous plot)
color_map = {
    '0': '#C7D9DD',
    '1': '#4A90E2', # highlight Largest
    '2': '#5A5A5A',
    '3': '#6BA292',
    '4+': '#7A86A1'
}

# Data prep
stay_counts = df['Stay_In_Current_City_Years'].value_counts().reindex(stay_order)
stay_perc = (stay_counts / stay_counts.sum() * 100).round(1)

stay_counts = stay_counts.dropna()
stay_perc = stay_perc.loc[stay_counts.index]

bar_colors = [color_map[val] for val in stay_counts.index]

# ----- Figure -----
fig, ax = plt.subplots(figsize=(17, 8))

bars = ax.bar(stay_counts.index, stay_counts.values,
              color=bar_colors, width=0.6, zorder=3)

# Labels (count + %)
for bar, val, pct in zip(bars, stay_counts.values, stay_perc.values):
    ax.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height() + 3000,
        f'{val}\n({pct:.1f}%)',
        ha='center',
        fontsize=10,
        fontfamily='serif'
    )

# Style
ax.grid(axis='y', linestyle='--', linewidth=0.8, alpha=0.4, zorder=0)

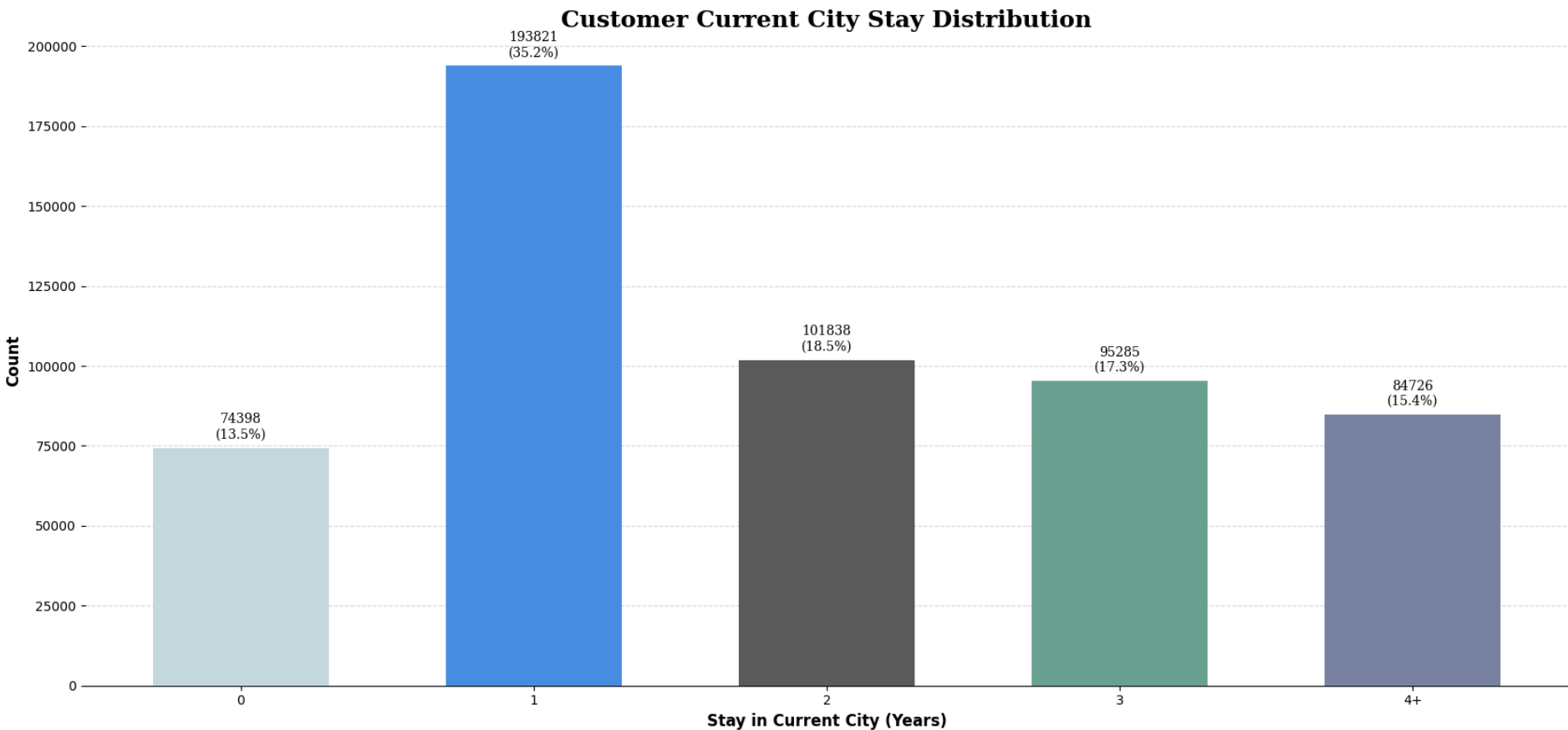
for s in ['top', 'right', 'left']:
```



```
ax.spines[s].set_visible(False)

ax.set_ylabel('Count', fontsize=12, fontweight='bold')
ax.set_xlabel('Stay in Current City (Years)', fontsize=12, fontweight='bold')
ax.set_title('Customer Current City Stay Distribution',
             fontsize=18, fontweight='bold', fontfamily='serif')

plt.tight_layout()
plt.show()
```



🔍 Insights

Customers staying for **1 year form the largest segment (35.2%)**, indicating that relatively new residents are the most active shoppers during Black Friday.

There is a **gradual decline in participation as stay duration increases**, suggesting that shorter-term residents are more engaged compared to long-term residents.

However, mid-term residents (2–3 years ~36%) still contribute significantly, showing that engagement remains strong beyond just newcomers.

The **4+ years group (15.4%)**, while smaller, represents a stable base of long-term customers, highlighting an opportunity to strengthen loyalty through targeted retention strategies.

3.2.4 🛒 Top 10 Products and Categories: Sales Snapshot

Top 10 Products and Product Categories which has sold most during Black Friday Sales

```
In [84]: fig, axes = plt.subplots(1, 2, figsize=(17, 7))

# Color setup
base_color = '#A6C8D0'
highlight_color = '#4A90E2'

def bar_chart(ax, series, title, xlabel, ylabel, text_offset):
    # Top 10 sorted
    temp = series.value_counts().head(10).sort_values()

    # Highlight top 3
    colors = [base_color] * (len(temp) - 3) + [highlight_color] * 3

    # Plot
    bars = ax.barh(temp.index.astype(str), temp.values, color=colors)

    # Labels
    for bar, val in zip(bars, temp.values):
        ax.text(val + text_offset,
                bar.get_y() + bar.get_height()/2,
                f'{val}',
                va='center',
                fontsize=10,
                fontfamily='serif')

    # Style
    ax.set_xlabel(xlabel, fontsize=12, fontweight='bold')
    ax.set_ylabel(ylabel, fontsize=12, fontweight='bold')
    ax.set_title(title, fontsize=15, fontweight='bold', fontfamily='serif')

    ax.grid(axis='x', linestyle='--', alpha=0.4)

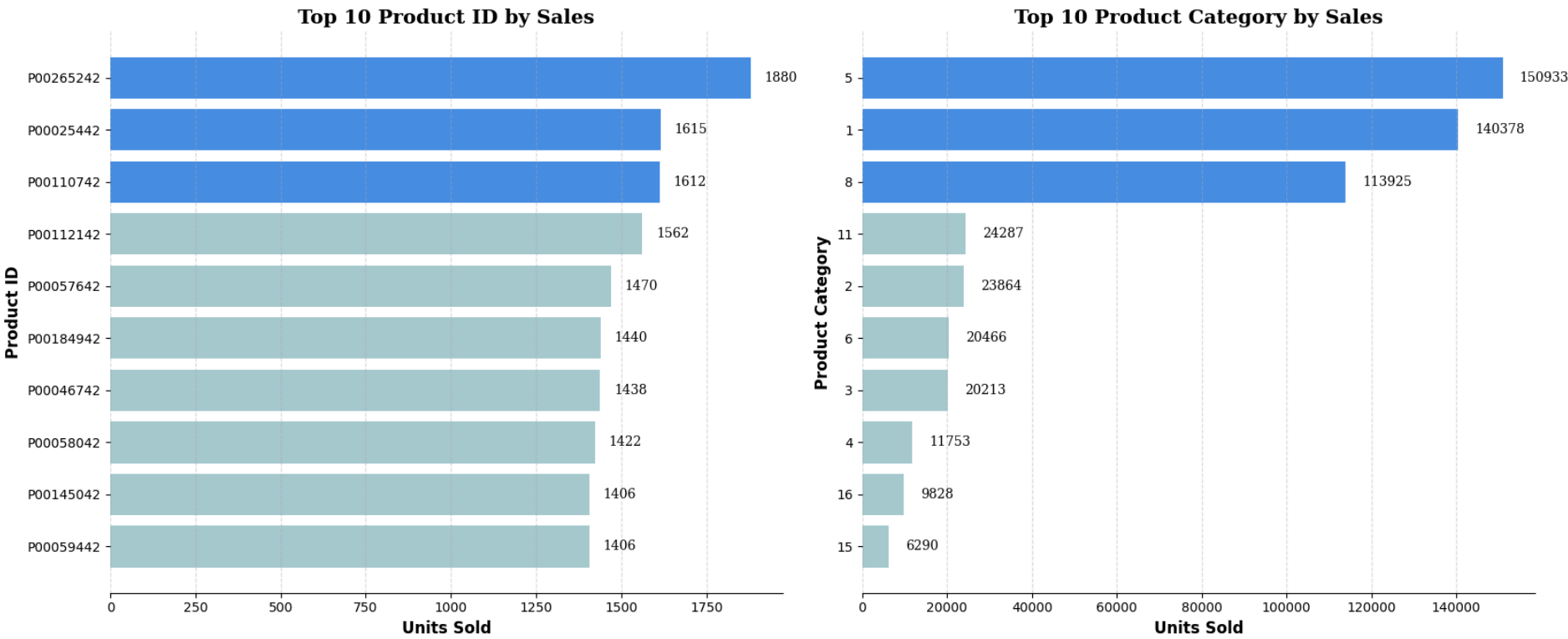
    for s in ['top', 'right', 'left']:
        ax.spines[s].set_visible(False)

# Left: Product ID
```

```
bar_chart(
    axes[0],
    df['Product_ID'],
    'Top 10 Product ID by Sales',
    'Units Sold',
    'Product ID',
    text_offset=40
)

# Right: Product Category
bar_chart(
    axes[1],
    df['Product_Category'],
    'Top 10 Product Category by Sales',
    'Units Sold',
    'Product Category',
    text_offset=4000
)

plt.tight_layout()
plt.show()
```



Insights:

For Product ID

The top-selling product IDs have relatively close unit sales, with the highest-selling product only slightly ahead of the others. This suggests that demand is distributed across multiple products rather than being dominated by a single item.

For Product Category

A few product categories contribute disproportionately to total unit sales, with categories 5, 1, and 8 leading by a large margin. This indicates that customer demand is heavily concentrated in specific categories during Black Friday.

3.2.5 🧑‍🏭 🧑‍🔧 Top 10 Customer Occupation

Top 10 Occupation of Customer in Black Friday Sales

```
In [85]: # Top 10 sorted
temp = df['Occupation'].value_counts().head(10).sort_values()

fig, ax = plt.subplots(figsize=(17, 7))

# Colors (highlight top 3)
colors = ['#A6C8D0'] * (len(temp) - 3) + ['#4A90E2'] * 3

# Plot
bars = ax.barh(temp.index.astype(str), temp.values, color=colors)

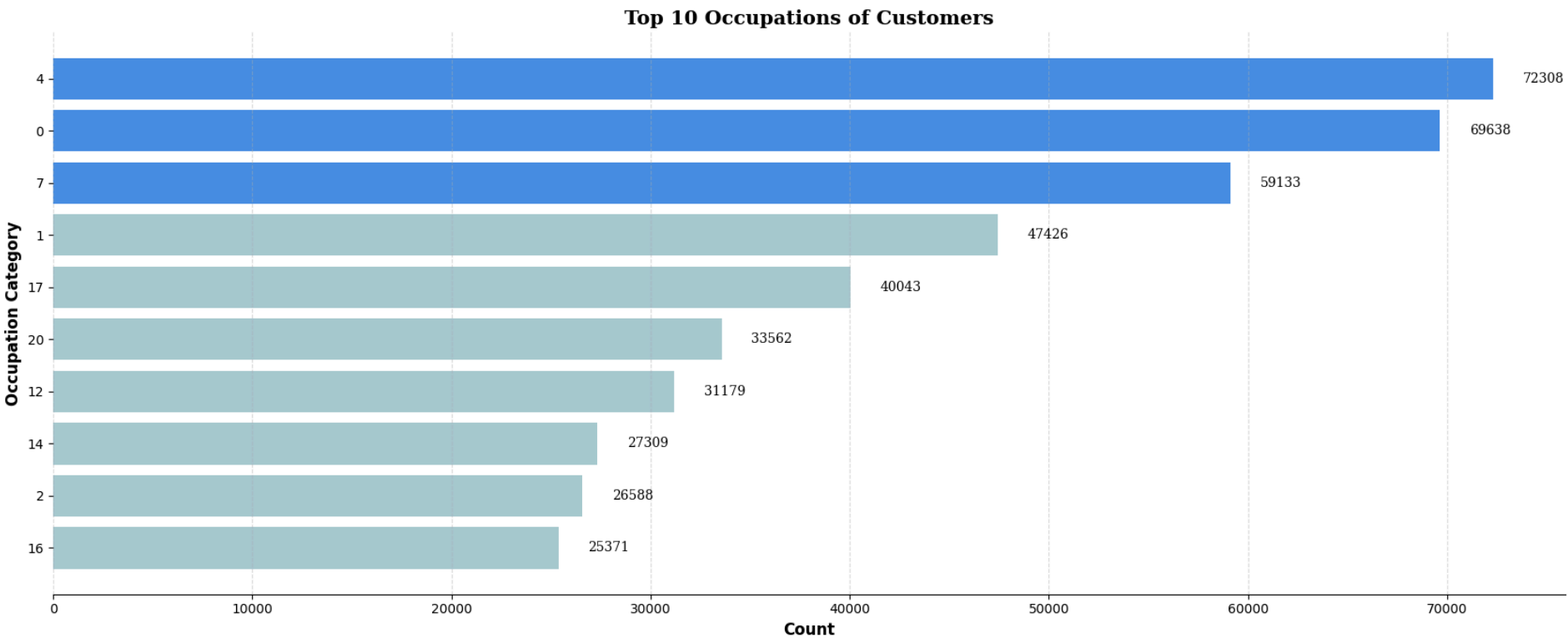
# Labels
for bar, val in zip(bars, temp.values):
    ax.text(val + 1500,
            bar.get_y() + bar.get_height()/2,
            f'{val}',
            va='center',
            fontsize=10,
            fontfamily='serif')

# Style
ax.set_xlabel('Count', fontsize=12, fontweight='bold')
ax.set_ylabel('Occupation Category', fontsize=12, fontweight='bold')
ax.set_title('Top 10 Occupations of Customers',
            fontsize=15, fontweight='bold', fontfamily='serif')

ax.grid(axis='x', linestyle='--', alpha=0.4)

# Clean Look
for s in ['top', 'right', 'left']:
    ax.spines[s].set_visible(False)
```

```
plt.tight_layout()
plt.show()
```



🔍 Insights

A few occupation categories (4, 0, 7) dominate the customer base, contributing significantly higher counts than others, indicating a strong concentration of shoppers within specific professional groups.

There is a steep drop after the top 3, suggesting that the remaining occupations contribute moderately and are more evenly distributed.

Mid-tier occupations (like 1 and 17) still show meaningful participation, indicating a broad but top-heavy customer distribution.

Overall, the distribution suggests that while Walmart attracts customers across various occupations, a small number of segments drive the majority of traffic, making them key targets for marketing and personalization strategies.

4.Bivariate Analysis

4.1 📊 Exploring Purchase Patterns

Boxplots of Purchase Amount Across various Variables

```
In [86]: # figure
fig, axes = plt.subplots(3, 2, figsize=(15, 20))
axes = axes.flatten()

cols = ['Gender', 'City_Category', 'Marital_Status',
        'Stay_In_Current_City_Years', 'Age']

palette = sns.color_palette("Set2")

for idx, col in enumerate(cols):
    ax = axes[idx]

    # ordered categories (VERY important)
    order = sorted(df[col].dropna().unique())

    sns.boxplot(
        data=df,
        x=col,
        y='Purchase',
        ax=ax,
        width=0.5,
        palette=palette,
        order=order
    )

    # titles + labels
    ax.set_title(f'Purchase vs {col}',
                fontsize=13, fontweight='bold', fontfamily='serif')

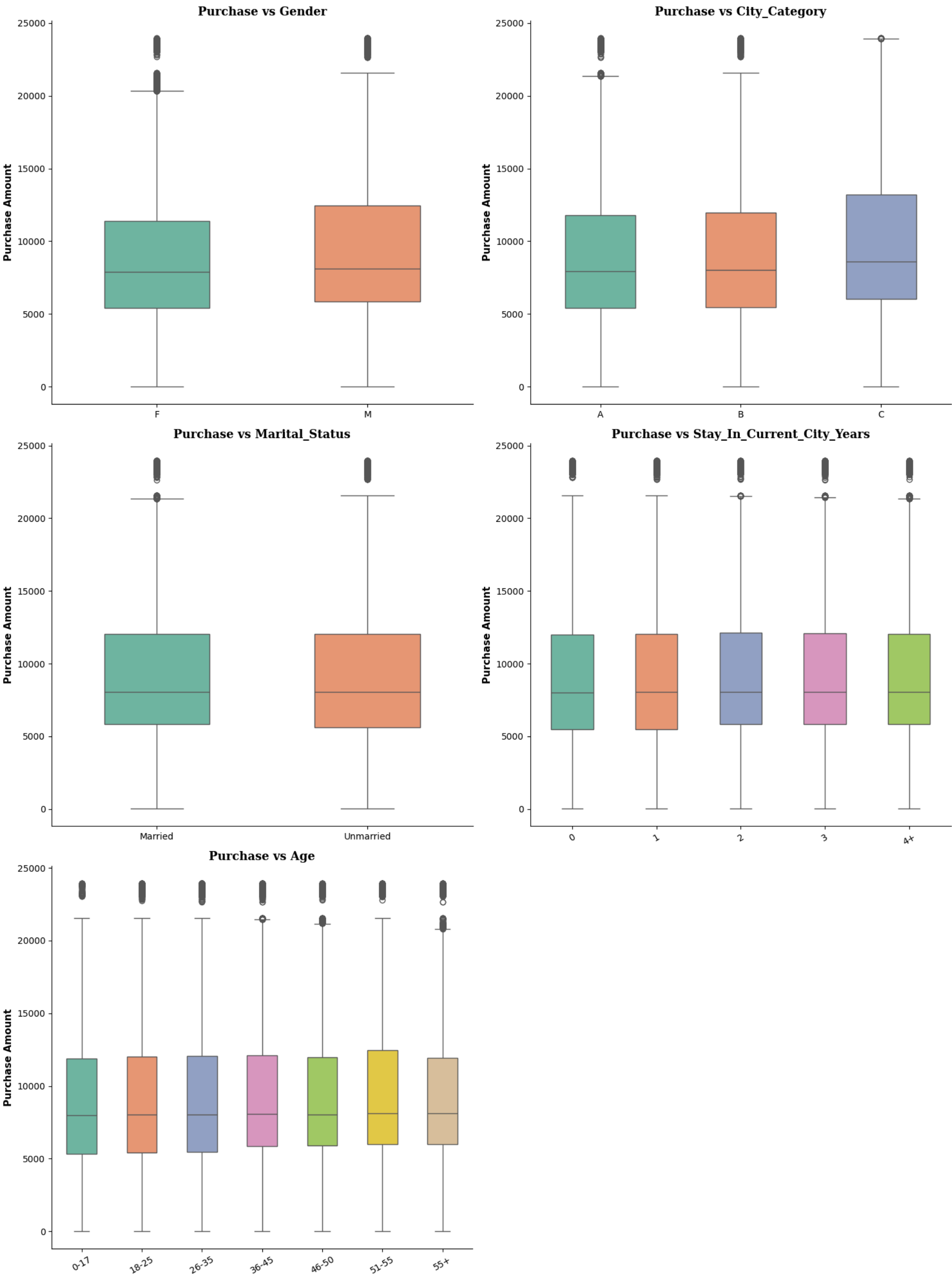
    ax.set_xlabel('')
    ax.set_ylabel('Purchase Amount', fontsize=11, fontweight='bold')

    # rotate if needed
    if col in ['Age', 'Stay_In_Current_City_Years']:
        ax.tick_params(axis='x', rotation=30)

    # clean look
    for s in ['top', 'right']:
        ax.spines[s].set_visible(False)

# remove empty subplot (last one)
fig.delaxes(axes[-1])

plt.tight_layout()
plt.show()
```



 Insights

- **Gender:** Purchase distribution is very similar across males and females, with only a slight increase in median spending for males.
- **City Category:** City C shows a slightly higher median and wider spread, indicating higher and more variable spending compared to A and B.
- **Marital Status:** Married and unmarried customers have nearly identical purchase behavior, with no significant difference in spending.
- **Stay Duration:** Purchase amounts remain fairly consistent across all stay durations, showing no strong impact of how long a customer has lived in the city.
- **Age:** Spending is relatively stable across age groups, with a slight increase in middle-aged groups (26–50), but no major variation overall.

Overall, purchase behavior remains largely consistent across all customer segments, with only minor variations, indicating no strong dependency of spending on demographic factors.

5. 🧑🏻 🧑🏼 Gender VS 💰 Purchase Amount

5.1 📊 Data Visualization

```
In [87]: #creating a df for purchase amount vs gender
temp = df.groupby('Gender')['Purchase'].agg(['sum', 'count']).reset_index()

#calculating the amount in billions
temp['sum_in_billions'] = round(temp['sum'] / 10**9,2)

#calculationg percentage distribution of purchase amount
temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)

#calculationg per purchase amount
temp['per_purchase'] = round(temp['sum']/temp['count'])

#renaming the gender
temp['Gender'] = temp['Gender'].replace({'F':'Female', 'M':'Male'})

temp
```

Out[87]:

	Gender	sum	count	sum_in_billions	%sum	per_purchase
0	Female	1186232642	135809	1.19	0.233	8735.0
1	Male	3909580100	414259	3.91	0.767	9438.0

```
In [89]: female_color = "#6C9BCF"    # soft blue
male_color = "#3D3D3D"             # charcoal
palette = [female_color, male_color]

fig = plt.figure(figsize=(15, 14))
gs = fig.add_gridspec(3, 2, height_ratios=[0.12, 0.38, 0.5])

# ----- Top stacked bar -----
ax0 = fig.add_subplot(gs[0, :])

g1, g2 = temp.loc[0, 'Gender'], temp.loc[1, 'Gender']
g1_pct, g2_pct = temp.loc[0, '%sum'], temp.loc[1, '%sum']
g1_bill, g2_bill = temp.loc[0, 'sum_in_billions'], temp.loc[1, 'sum_in_billions']

ax0.barh([''], g1_pct, color=female_color)
ax0.barh([''], g2_pct, left=g1_pct, color=male_color)

ax0.text(g1_pct/2, 0, f"$ {g1_bill}B\n{g1}",
        ha='center', va='center', color='white', fontsize=14, fontweight='bold')

ax0.text(g1_pct + g2_pct/2, 0, f"$ {g2_bill}B\n{g2}",
        ha='center', va='center', color='white', fontsize=14, fontweight='bold')

ax0.set_xlim(0,1)
ax0.set_xticks([])
ax0.set_yticks([])
for s in ['top', 'left', 'right', 'bottom']:
    ax0.spines[s].set_visible(False)

ax0.set_title('Total Purchase Contribution by Gender',
             fontsize=15, fontweight='bold', fontfamily='serif')

# ----- Avg purchase -----
ax1 = fig.add_subplot(gs[1,0])

bars = ax1.bar(temp['Gender'], temp['per_purchase'],
              color=palette, width=0.4)

avg = df['Purchase'].mean()
ax1.axhline(avg, color='#E63946', linestyle='--')

ax1.text(0.5, avg+200, f"Overall Avg = ${avg:.0f}",
        ha='center', fontsize=11)

for bar, val in zip(bars, temp['per_purchase']):
    ax1.text(bar.get_x()+bar.get_width()/2, val/2,
            f"${val:.0f}", ha='center',
            color='white', fontweight='bold')

ax1.set_title('Avg Purchase per Transaction',
             fontsize=15, fontweight='bold', fontfamily='serif')

ax1.grid(axis='y', alpha=0.3)
for s in ['top', 'right', 'left']:
    ax1.spines[s].set_visible(False)

# ----- Donut chart -----
ax2 = fig.add_subplot(gs[1,1])

ax2.pie(
    temp['count'],
    labels=temp['Gender'],
    autopct='%1.1f%%',
    colors=palette,
    startangle=90,
```

```
wedgeprops=dict(width=0.4)
)

ax2.set_title('Transaction Share by Gender',
              fontsize=15, fontweight='bold', fontfamily='serif')

# ----- KDE plot (kept, just improved colors) -----
ax3 = fig.add_subplot(gs[2,:])

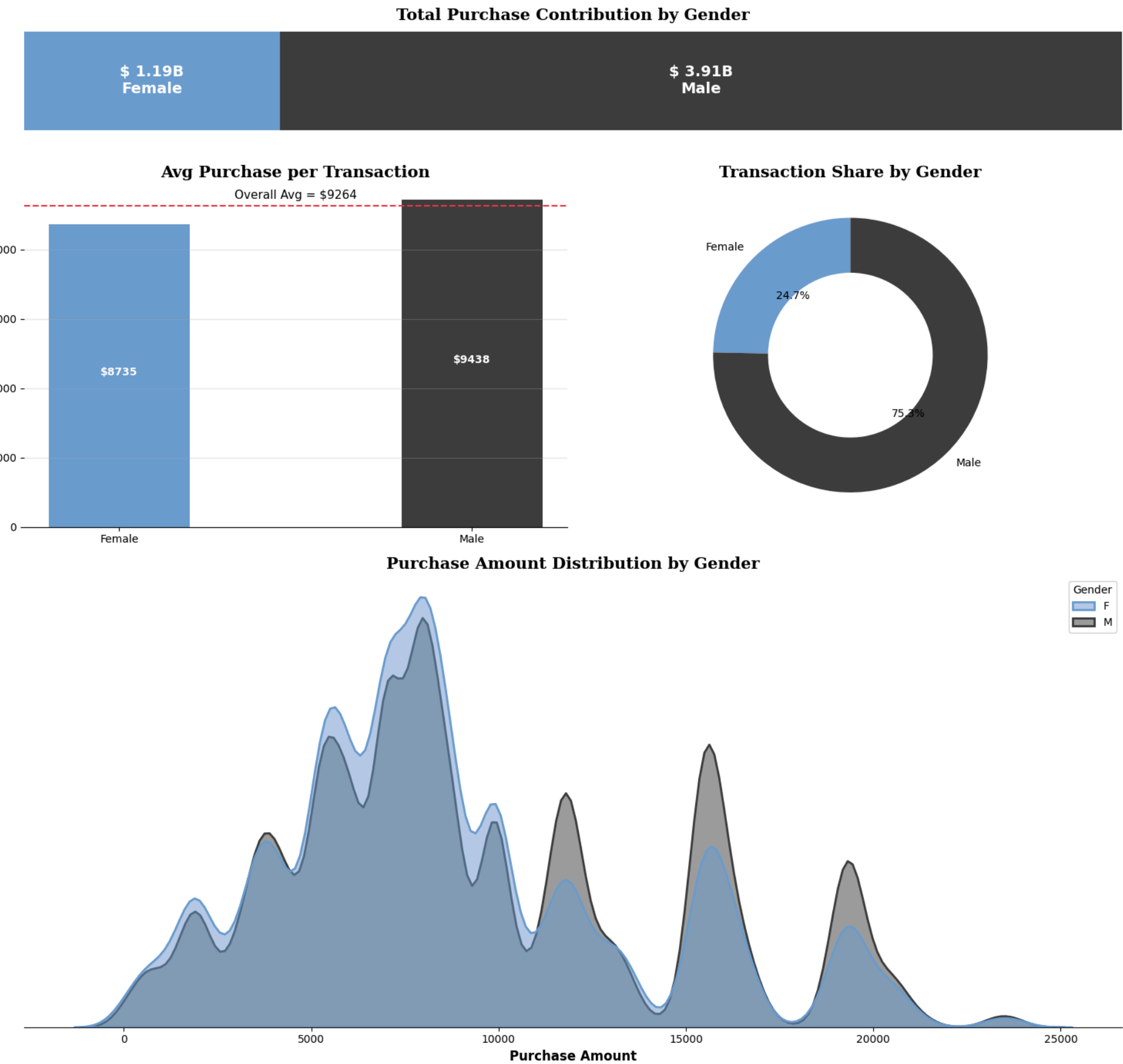
sns.kdeplot(
    data=df,
    x='Purchase',
    hue='Gender',
    palette=palette,
    fill=True,
    alpha=0.5,           # softer fill
    linewidth=2,
    common_norm=False,   # important for fair comparison
    ax=ax3
)

for s in ['top','left','right']:
    ax3.spines[s].set_visible(False)

ax3.set_yticks([])
ax3.set_ylabel('')
ax3.set_xlabel('Purchase Amount', fontsize=12, fontweight='bold')

ax3.set_title('Purchase Amount Distribution by Gender',
              fontsize=15, fontweight='bold', fontfamily='serif')

plt.tight_layout()
plt.show()
```



🔍 Insights

- 1. **Total Sales & Transactions** Male customers contribute significantly more to total sales and transactions (over 3× higher), indicating that overall revenue is primarily driven by higher participation from male shoppers.

2. **Average Transaction Value** The average purchase per transaction is only slightly higher for males (9438*vs*8735), suggesting that the difference in total sales is mainly due to transaction volume rather than spending per purchase.
3. **Purchase Distribution** The purchase distributions for both genders are similarly shaped and non-normal, indicating comparable spending patterns with the presence of high-value outliers in both groups.

5.2 Confidence Interval Construction: Estimating Average Purchase Amount per Transaction

1. Step 1 - Building CLT Curve

- As seen above, the purchase amount distribution is not Normal. So we need to use Central Limit Theorem. It states the distribution of sample means will approximate a normal distribution, regardless of the underlying population distribution

2. Step 2 - Building Confidence Interval

- After building CLT curve, we will create a confidence interval predicting population mean at 99%,95% and 90% Confidence level.

Note - We will use different sample sizes of [100,1000,5000,50000]

```
In [90]: #creating a function to calculate confidence interval

def confidence_interval(data,ci):
    #converting the list to series
    l_ci = (100-ci)/2
    u_ci = (100+ci)/2

    #calculating lower limit and upper limit of confidence interval
    interval = np.percentile(data,[l_ci,u_ci]).round(0)

    return interval
```

```
In [93]: def plot(ci):

    fig = plt.figure(figsize=(15, 8))
    gs = fig.add_gridspec(2, 2)

    male_color = "#2A9D8F"    # teal
    female_color = "#E76F51"  # coral

    df_male = df.loc[df['Gender'] == 'M', 'Purchase']
    df_female = df.loc[df['Gender'] == 'F', 'Purchase']

    # 🔄 shuffled layout (not ordered)
    sample_sizes = [
        (5000, 0, 0),
        (100, 0, 1),
        (50000, 1, 0),
        (1000, 1, 1)
    ]

    bootstrap_samples = 20000

    male_samples = {}
    female_samples = {}

    for size, row, col in sample_sizes:

        male_means = []
        female_means = []

        for _ in range(bootstrap_samples):

            male_sample = np.random.choice(df_male, size=size)
            female_sample = np.random.choice(df_female, size=size)

            male_means.append(np.mean(male_sample))
            female_means.append(np.mean(female_sample))

        male_samples[f'{ci}%_{size}'] = male_means
        female_samples[f'{ci}%_{size}'] = female_means

        temp_df = pd.DataFrame({
            'Male': male_means,
            'Female': female_means
        })

        ax = fig.add_subplot(gs[row, col])

        # KDE plots
        sns.kdeplot(temp_df['Male'], color=male_color, fill=True,
                    alpha=0.4, ax=ax, label='Male', linewidth=2)

        sns.kdeplot(temp_df['Female'], color=female_color, fill=True,
                    alpha=0.4, ax=ax, label='Female', linewidth=2)

        # Confidence intervals
        m_range = confidence_interval(male_means, ci)
        f_range = confidence_interval(female_means, ci)

        for k in m_range:
            ax.axvline(x=k, color=male_color, linestyle='--', linewidth=1.5)
```



```
for k in f_range:
    ax.axvline(x=k, color=female_color, linestyle='--', linewidth=1.5)

# Styling
for s in ['top', 'left', 'right']:
    ax.spines[s].set_visible(False)

ax.set_yticks([])
ax.set_xlabel('')
ax.set_ylabel('')

ax.set_title(f'Sample Size = {size}',
             fontdict={'family': 'serif', 'size': 12, 'weight': 'bold'})

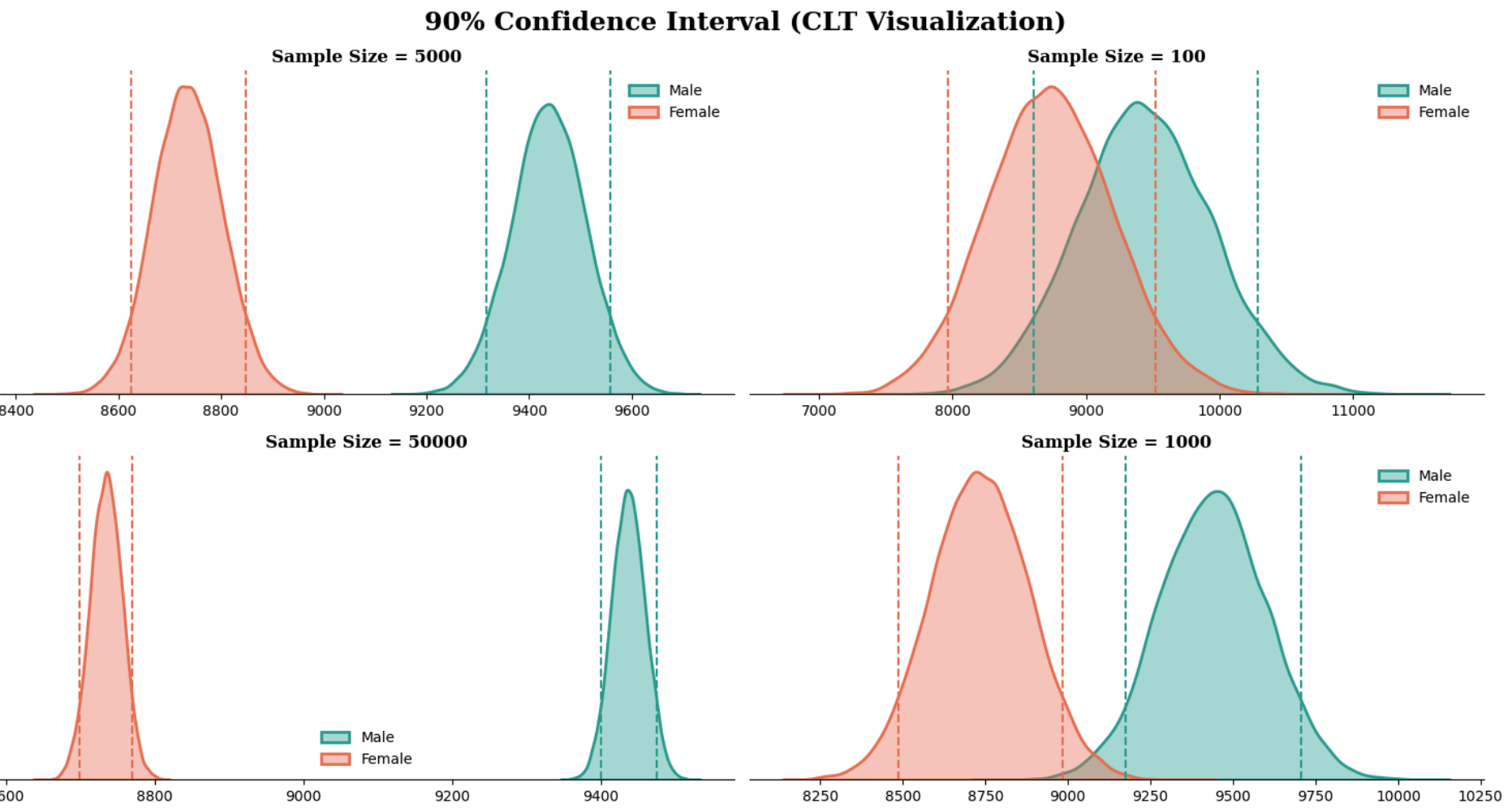
ax.legend(frameon=False)

fig.suptitle(f'{ci}% Confidence Interval (CLT Visualization)',
            fontsize=18, fontweight='bold', fontfamily='serif')

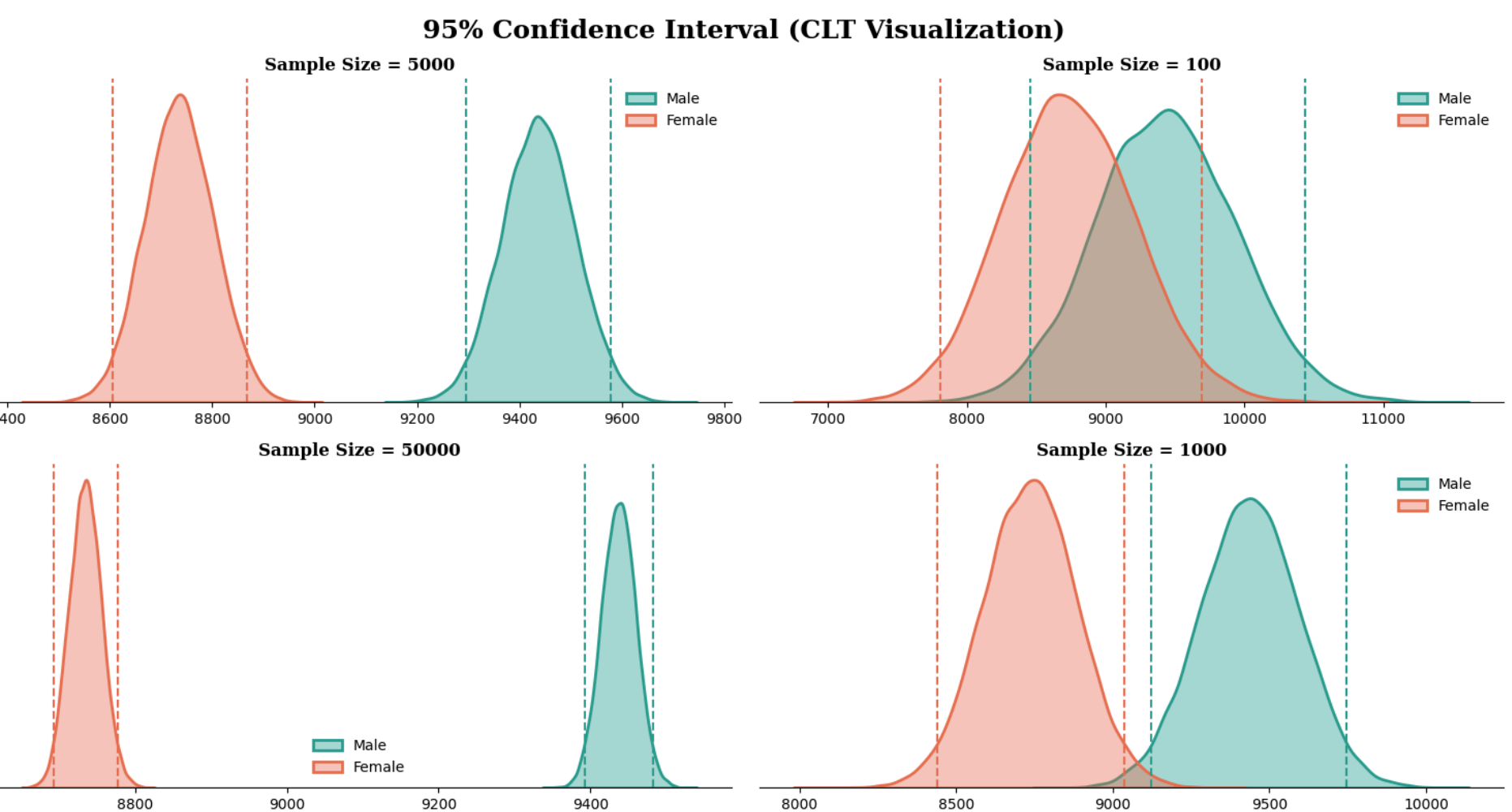
plt.tight_layout()
plt.show()

return male_samples, female_samples

m_samp_90, f_samp_90 = plot(90)
```

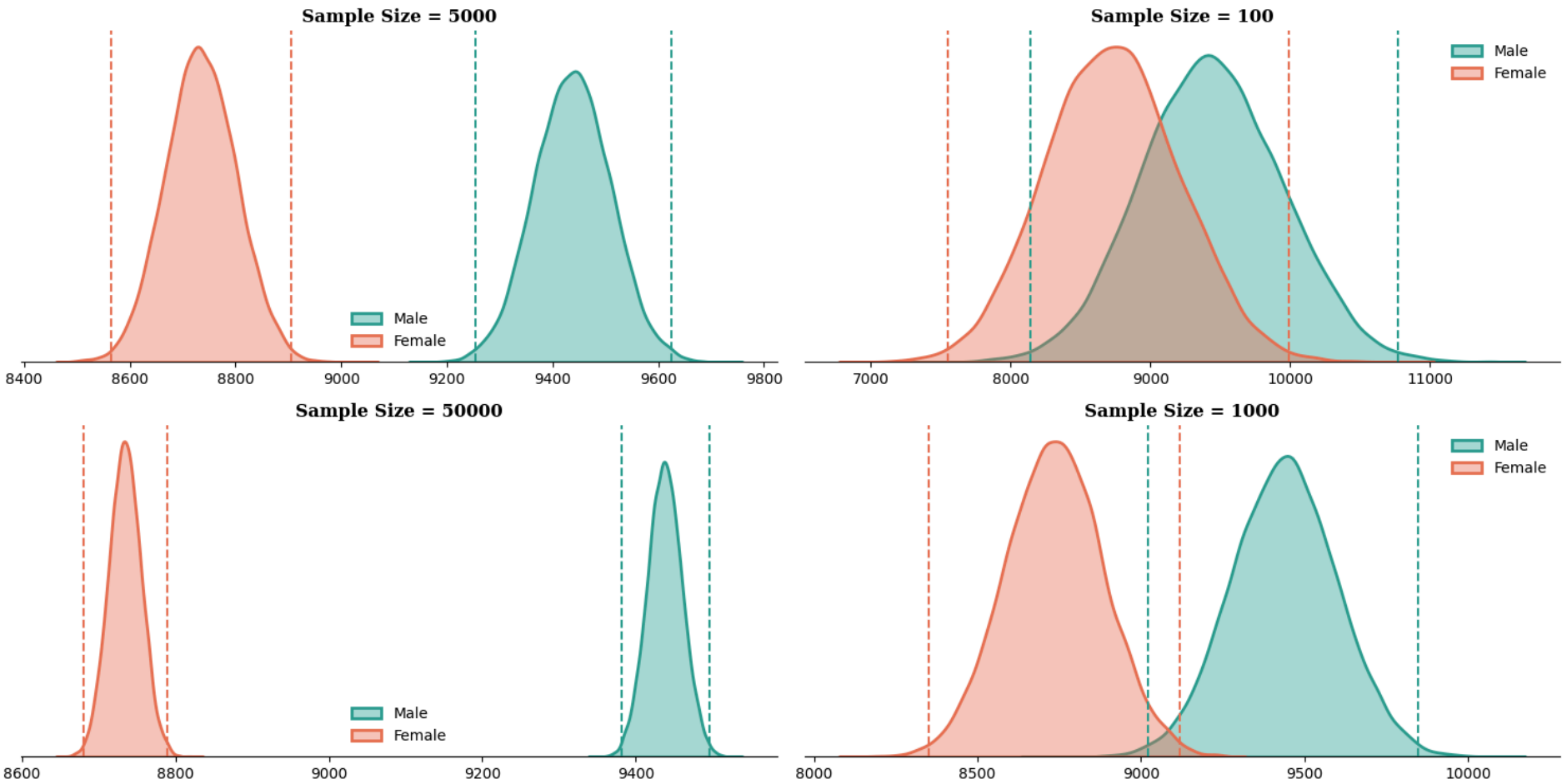


```
In [94]: m_samp_95,f_samp_95 = plot(95)
```



```
In [95]: m_samp_99,f_samp_99 = plot(99)
```


99% Confidence Interval (CLT Visualization)



Are confidence intervals of average male and female spending overlapping?

In [100...

```
def print_ci_overlap_summary(male_dicts, female_dicts, confidence_levels):
    import re

    for m_dict, f_dict, cl in zip(male_dicts, female_dicts, confidence_levels):
        print(f'\n{cl}% Confidence Interval Overlap Summary')
        print('-' * 45)

        sorted_keys = sorted(m_dict.keys(), key=lambda x: int(re.search(r'_(\d+)', x).group(1)))

        for key in sorted_keys:
            size = int(re.search(r'_(\d+)', key).group(1))

            m_ci = confidence_interval(m_dict[key], cl)
            f_ci = confidence_interval(f_dict[key], cl)

            overlap = not (m_ci[1] < f_ci[0] or f_ci[1] < m_ci[0])

            print(f'Sample size {size}:')
            print(f'  Male CI   = ({m_ci[0]:.2f}, {m_ci[1]:.2f})')
            print(f'  Female CI = ({f_ci[0]:.2f}, {f_ci[1]:.2f})')
            print(f'  Overlap?  = {"Yes" if overlap else "No"}\n')

print_ci_overlap_summary(
    [m_samp_90, m_samp_95, m_samp_99],
    [f_samp_90, f_samp_95, f_samp_99],
    [90, 95, 99]
)
```

90% Confidence Interval Overlap Summary

Sample size 100:
Male CI = (\$8611.00, \$10289.00)
Female CI = (\$7963.00, \$9522.00)
Overlap? = Yes

Sample size 1000:
Male CI = (\$9174.00, \$9707.00)
Female CI = (\$8487.00, \$8984.00)
Overlap? = No

Sample size 5000:
Male CI = (\$9317.00, \$9558.00)
Female CI = (\$8625.00, \$8848.00)
Overlap? = No

Sample size 50000:
Male CI = (\$9400.00, \$9475.00)
Female CI = (\$8699.00, \$8770.00)
Overlap? = No

95% Confidence Interval Overlap Summary

Sample size 100:
Male CI = (\$8458.00, \$10433.00)
Female CI = (\$7810.00, \$9690.00)
Overlap? = Yes

Sample size 1000:
Male CI = (\$9123.00, \$9747.00)
Female CI = (\$8440.00, \$9036.00)
Overlap? = No

Sample size 5000:
Male CI = (\$9296.00, \$9579.00)
Female CI = (\$8605.00, \$8867.00)
Overlap? = No

Sample size 50000:
Male CI = (\$9393.00, \$9482.00)
Female CI = (\$8693.00, \$8777.00)
Overlap? = No

99% Confidence Interval Overlap Summary

Sample size 100:
Male CI = (\$8142.00, \$10768.00)
Female CI = (\$7547.00, \$9988.00)
Overlap? = Yes

Sample size 1000:
Male CI = (\$9021.00, \$9848.00)
Female CI = (\$8351.00, \$9119.00)
Overlap? = Yes

Sample size 5000:
Male CI = (\$9254.00, \$9625.00)
Female CI = (\$8565.00, \$8906.00)
Overlap? = No

Sample size 50000:
Male CI = (\$9381.00, \$9496.00)
Female CI = (\$8681.00, \$8790.00)
Overlap? = No

 Insights

- At **small sample sizes (n = 100)**, confidence intervals overlap across all confidence levels, indicating **high uncertainty** and no clear difference in average spending.
- As sample size increases (**n ≥ 1000**), intervals **separate clearly**, especially for 90% and 95% CIs, showing a **consistent higher average spending by males**.
- Even at **99% confidence**, overlap disappears at larger sample sizes (≥ 5000), reinforcing that the difference is **statistically meaningful**, not due to randomness.
- The **gap between male and female means remains stable (~700–800)**, while intervals become tighter, indicating **increasing precision with larger samples**.

As sample size increases, confidence intervals narrow and separate, providing strong evidence that males have a consistently higher average spending than females.

5. How Walmart Can Leverage This Insight

5.1 Segmentation Opportunities

Walmart can design **targeted campaigns and product bundles** for male and female customers, leveraging their distinct spending patterns to improve engagement and maximize revenue from each segment.

5.2 Pricing Strategies

Since males show a slightly higher average spend, Walmart can use **upselling and premium pricing strategies** for male-focused products, while offering **value-driven pricing and promotions** for female-oriented segments to maintain competitiveness and volume.

Note: Further analysis will be conducted using a **95% Confidence Level** for consistency and reliability.

6. 👤 Marital Status VS 💰 Purchase Amount

6.1 📊 Data Visualization

In [101...

```
#creating a df for purchase amount vs marital status
temp = df.groupby('Marital_Status')['Purchase'].agg(['sum','count']).reset_index()

#calculating the amount in billions
temp['sum_in_billions'] = round(temp['sum'] / 10**9,2)

#calculating percentage distribution of purchase amount
temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)

#calculating per purchase amount
temp['per_purchase'] = round(temp['sum']/temp['count'])

temp
```

Out[101...

	Marital_Status	sum	count	sum_in_billions	%sum	per_purchase
0	Unmarried	3008927447	324731	3.01	0.59	9266.0
1	Married	2086885295	225337	2.09	0.41	9261.0

In [104...

```
unmarried_color = "#6C9BCF" # blue
married_color = "#3D3D3D" # charcoal
palette = [unmarried_color, married_color]

# safer extraction
temp = temp.reset_index(drop=True)

g1 = temp.loc[0, 'Marital_Status']
g2 = temp.loc[1, 'Marital_Status']

g1_pct = temp.loc[0, '%sum']
g2_pct = temp.loc[1, '%sum']

g1_bill = temp.loc[0, 'sum_in_billions']
g2_bill = temp.loc[1, 'sum_in_billions']

avg = df['Purchase'].mean()

fig = plt.figure(figsize=(15, 14))
gs = fig.add_gridspec(3, 2, height_ratios=[0.12, 0.38, 0.5])

# ----- Top stacked bar -----
ax0 = fig.add_subplot(gs[0, :])

ax0.barh([''], g1_pct, color=unmarried_color)
ax0.barh([''], g2_pct, left=g1_pct, color=married_color)

ax0.text(g1_pct/2, 0, f"$ {g1_bill}B\n{g1}",
        ha='center', va='center', color='white', fontsize=14, fontweight='bold')

ax0.text(g1_pct + g2_pct/2, 0, f"$ {g2_bill}B\n{g2}",
        ha='center', va='center', color='white', fontsize=14, fontweight='bold')

for s in ['top','left','right','bottom']:
    ax0.spines[s].set_visible(False)

ax0.set_xticks([])
ax0.set_yticks([])
ax0.set_xlim(0,1)

ax0.set_title('Total Purchase Contribution by Marital Status',
             fontsize=15, fontweight='bold', fontfamily='serif')

# ----- Avg purchase -----
ax1 = fig.add_subplot(gs[1,0])

bars = ax1.bar(temp['Marital_Status'], temp['per_purchase'],
              color=palette, width=0.4)

ax1.axhline(avg, color='#E63946', linestyle='--')

ax1.text(0.5, avg+200, f"Overall Avg = ${avg:.0f}",
        ha='center', fontsize=11)
```

```
for bar, val in zip(bars, temp['per_purchase']):
    ax1.text(bar.get_x()+bar.get_width()/2, val/2,
             f"${val:.0f}", ha='center',
             color='white', fontweight='bold')

ax1.grid(axis='y', alpha=0.3)

for s in ['top', 'right', 'left']:
    ax1.spines[s].set_visible(False)

ax1.set_ylabel('Purchase Amount', fontsize=12, fontweight='bold')
ax1.set_title('Avg Purchase per Transaction',
              fontsize=15, fontweight='bold', fontfamily='serif')

# ----- Donut chart -----
ax2 = fig.add_subplot(gs[1,1])

ax2.pie(
    temp['count'],
    labels=temp['Marital_Status'],
    autopct='%1.1f%%',
    colors=palette,
    startangle=90,
    wedgeprops=dict(width=0.4)
)

ax2.set_title('Transaction Share by Marital Status',
              fontsize=15, fontweight='bold', fontfamily='serif')

# ----- KDE -----
ax3 = fig.add_subplot(gs[2,:])

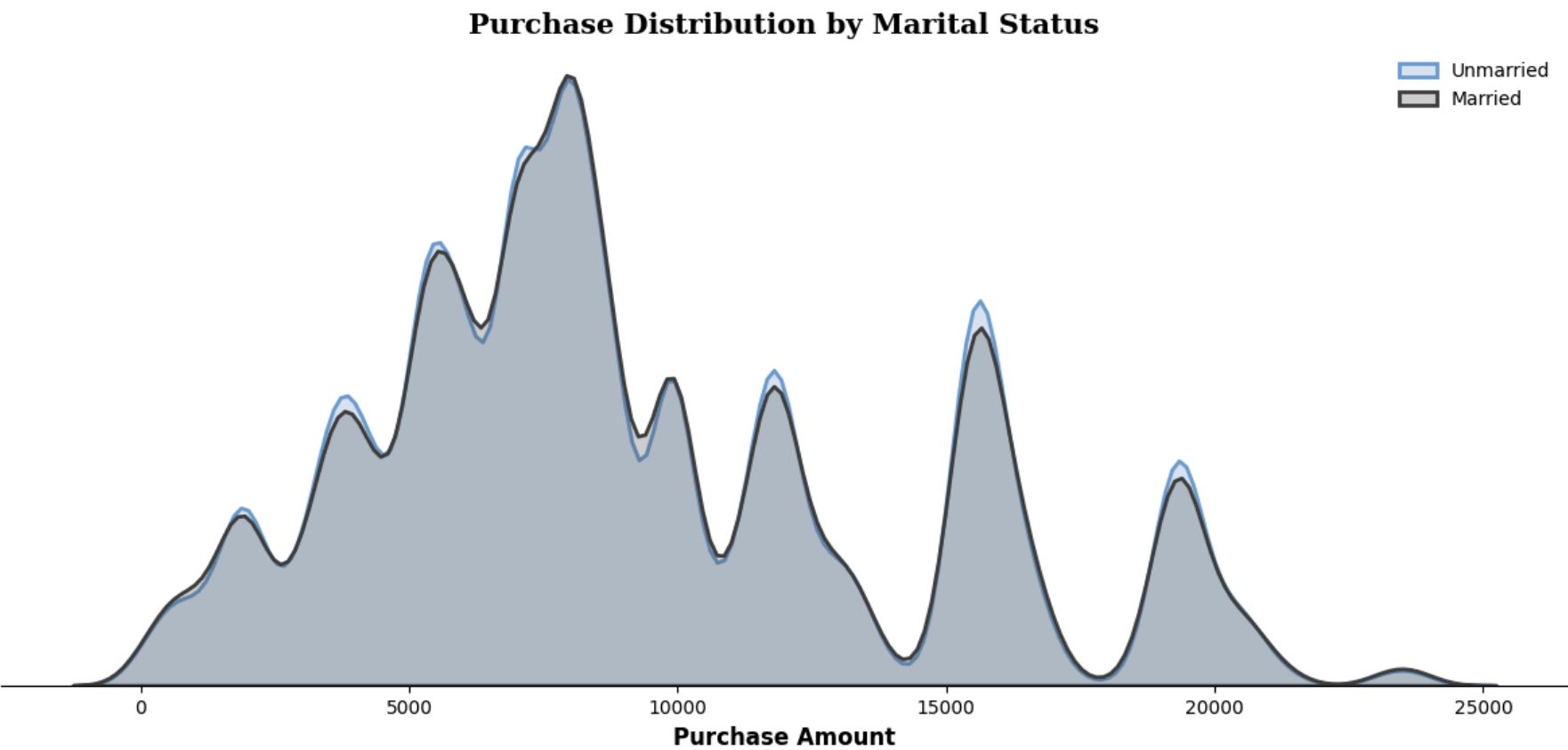
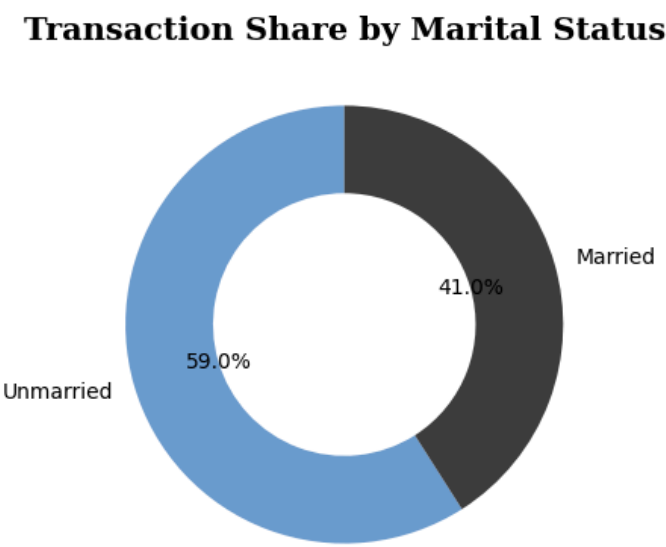
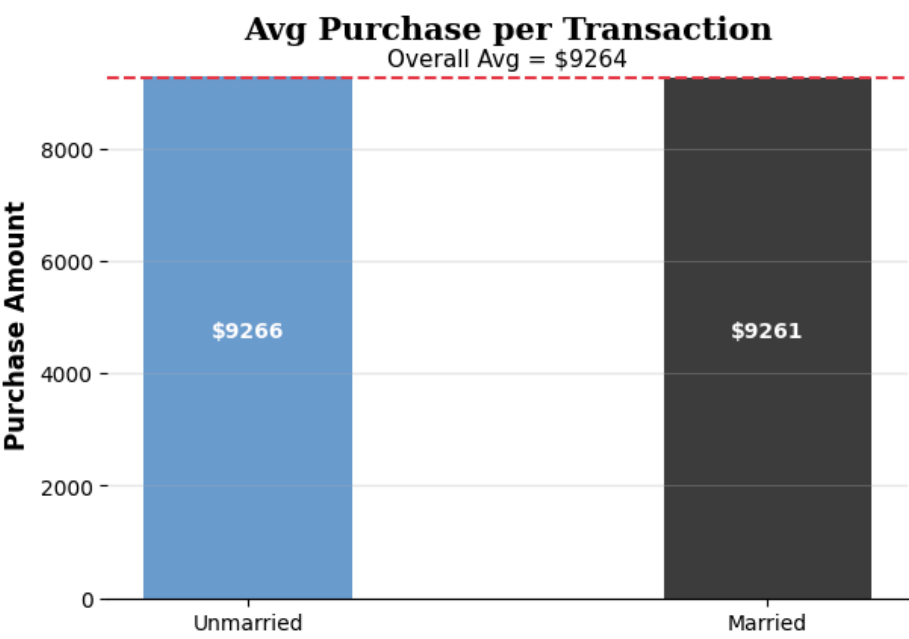
sns.kdeplot(
    data=df[df['Marital_Status'] == 'Unmarried'],
    x='Purchase',
    color=unmarried_color,
    fill=True,
    alpha=0.25,
    linewidth=2,
    ax=ax3,
    label='Unmarried'
)

sns.kdeplot(
    data=df[df['Marital_Status'] == 'Married'],
    x='Purchase',
    color=married_color,
    fill=True,
    alpha=0.25,
    linewidth=2,
    ax=ax3,
    label='Married'
)

for s in ['top', 'left', 'right']:
    ax3.spines[s].set_visible(False)

ax3.set_yticks([])
ax3.set_ylabel('')
ax3.set_xlabel('Purchase Amount', fontsize=12, fontweight='bold')
ax3.set_title('Purchase Distribution by Marital Status',
              fontsize=15, fontweight='bold', fontfamily='serif')

ax3.legend(frameon=False)
```



Here are clean, sharp insights for your **Marital Status dashboard**:

- Insights**
- Total Contribution:** Unmarried customers contribute a larger share of total revenue (~59%), mainly due to higher transaction volume.
 - Average Spending:** The average purchase per transaction is almost identical for both groups (~\$9260), indicating **no meaningful difference in spending per purchase**.
 - Distribution Pattern:** Purchase distributions for married and unmarried customers are highly similar, with comparable spread and outliers.

Marital status does not significantly influence spending behavior; differences in total sales are driven by the number of transactions rather than purchase amount.

6.2 Confidence Interval Construction: Estimating Average Purchase Amount per Transaction

1. Step 1 - Building CLT Curve

- As seen above, the purchase amount distribution is not Normal. So we need to use Central Limit Theorem. It states the distribution of sample means will approximate a normal distribution, regardless of the underlying population distribution

2. Step 2 - Building Confidence Interval

- After building CLT curve, we will create a confidence interval predicting population mean at 95% Confidence level.

Note - We will use different sample sizes of [100,1000,5000,50000]

```
In [107... import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

def plot(ci):

    fig = plt.figure(figsize=(15, 8))
    gs = fig.add_gridspec(2, 2)

    # separate data
```

```
df_married = df.loc[df['Marital_Status'] == 'Married', 'Purchase']
df_unmarried = df.loc[df['Marital_Status'] == 'Unmarried', 'Purchase']

# sample sizes
sample_sizes = [(100, 0, 0), (1000, 0, 1), (5000, 1, 0), (50000, 1, 1)]

bootstrap_samples = 20000

married_samples = {}
unmarried_samples = {}

# better colors
married_color = "#2A9D8F"
unmarried_color = "#E76F51"

for size, row, col in sample_sizes:
    married_means = []
    unmarried_means = []

    for _ in range(bootstrap_samples):
        married_sample = np.random.choice(df_married, size=size)
        unmarried_sample = np.random.choice(df_unmarried, size=size)

        married_means.append(np.mean(married_sample))
        unmarried_means.append(np.mean(unmarried_sample))

    married_samples[f'{ci}%_{size}'] = married_means
    unmarried_samples[f'{ci}%_{size}'] = unmarried_means

    temp_df = pd.DataFrame({
        'Married': married_means,
        'Unmarried': unmarried_means
    })

    ax = fig.add_subplot(gs[row, col])

    sns.kdeplot(
        data=temp_df, x='Married',
        color=married_color, fill=True, alpha=0.35,
        ax=ax, label='Married', linewidth=2
    )

    sns.kdeplot(
        data=temp_df, x='Unmarried',
        color=unmarried_color, fill=True, alpha=0.35,
        ax=ax, label='Unmarried', linewidth=2
    )

    # confidence intervals
    married_ci = confidence_interval(married_means, ci)
    unmarried_ci = confidence_interval(unmarried_means, ci)

    for x in married_ci:
        ax.axvline(x=x, ymax=0.9, color=married_color, linestyle='--', linewidth=1.5)

    for x in unmarried_ci:
        ax.axvline(x=x, ymax=0.9, color=unmarried_color, linestyle='--', linewidth=1.5)

    # styling
    for s in ['top', 'left', 'right']:
        ax.spines[s].set_visible(False)

    ax.set_yticks([])
    ax.set_ylabel('')
    ax.set_xlabel('')
    ax.set_title(f'CLT Curve for Sample Size = {size}',
                 fontdict={'family': 'serif', 'size': 11, 'weight': 'bold'})
    ax.legend(frameon=False)

fig.suptitle(f'{ci}% Confidence Interval',
            font='serif', size=18, weight='bold')

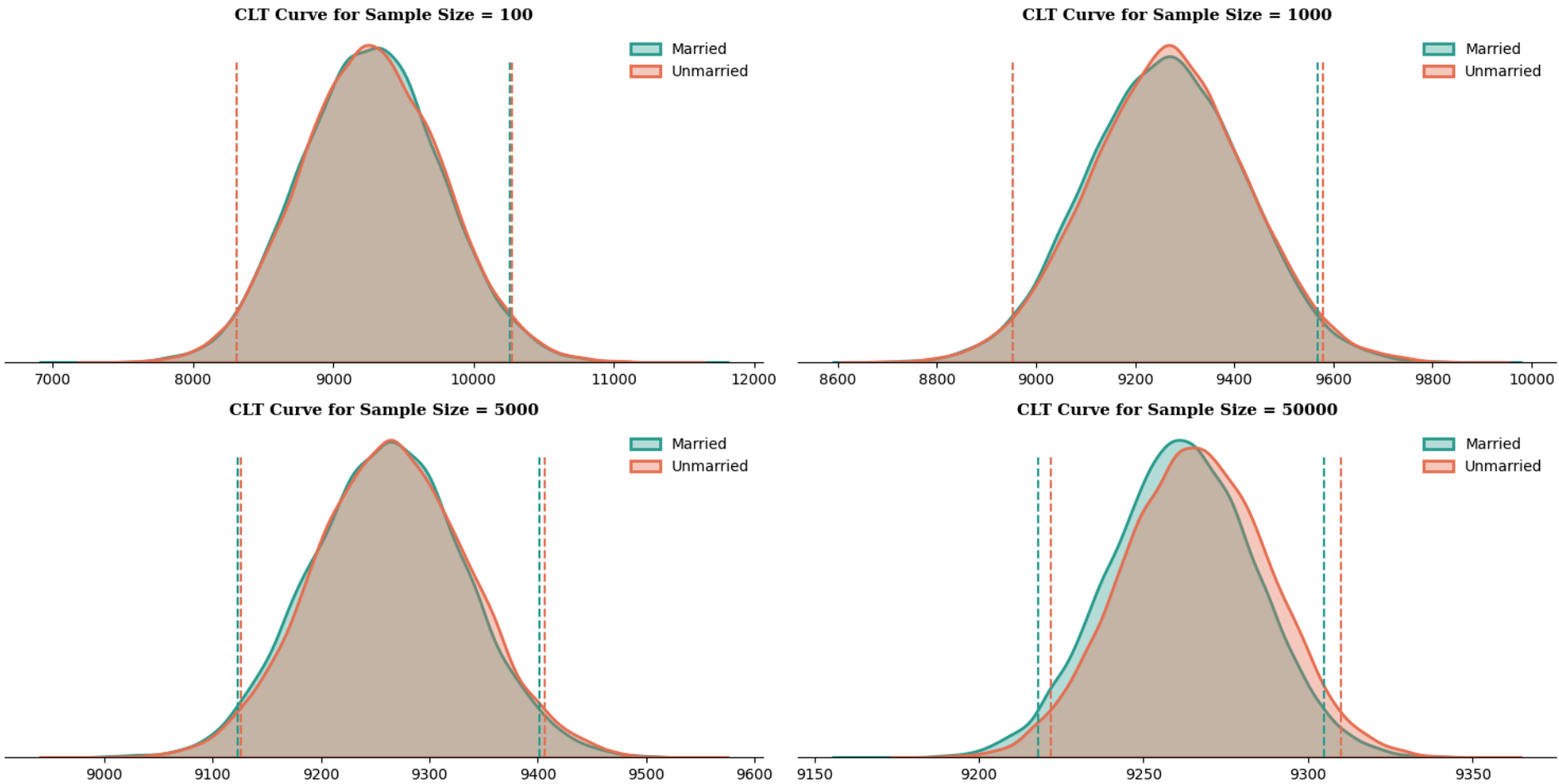
plt.tight_layout()
plt.show()

return married_samples, unmarried_samples
```

In [108...

m_samp_95,u_samp_95 = plot(95)

95% Confidence Interval



Are confidence intervals of average married and unmarried customer spending overlapping?

```
In [109... print("\n95% CI Overlap Summary\n" + "-"*40)

for key in sorted(m_samp_95, key=lambda x: int(x.split('_')[1])):
    size = key.split('_')[1]

    m_low, m_high = confidence_interval(m_samp_95[key], 95)
    u_low, u_high = confidence_interval(u_samp_95[key], 95)

    overlap = not (m_high < u_low or u_high < m_low)

    print(f"n={size}: "
          f"M=({m_low:.0f},{m_high:.0f}) | "
          f"U=({u_low:.0f},{u_high:.0f}) | "
          f"Overlap={'Y' if overlap else 'N'}")
```

```
95% CI Overlap Summary
-----
n=100: M=(8310,10261) | U=(8314,10270) | Overlap=Y
n=1000: M=(8954,9569) | U=(8954,9579) | Overlap=Y
n=5000: M=(9123,9402) | U=(9126,9407) | Overlap=Y
n=50000: M=(9218,9305) | U=(9222,9310) | Overlap=Y
```

🔍 Insights

- The **confidence intervals overlap at all sample sizes**, indicating **no meaningful difference** in average spending between married and unmarried customers.
- As sample size increases, the intervals become **narrower and almost identical**, showing **high precision and strong consistency** in the estimates.
- The **mean purchase ranges are nearly the same** for both groups across all samples, reinforcing that marital status has **minimal impact on spending behavior**.

There is no statistically significant difference in average spending between married and unmarried customers, even at high precision levels.

How can Walmart leverage this conclusion to make changes or improvements?

Marketing Resources

Walmart may not need to allocate marketing resources specifically targeting one group over the other. Instead, they can focus on broader marketing strategies that appeal to both groups.

7. 📅 Customer Age VS 💰 Purchase Amount

7.1 📊 Data Visualization

```
In [110... #creating a df for purchase amount vs age group
temp = df.groupby('Age')['Purchase'].agg(['sum', 'count']).reset_index()

#calculating the amount in billions
temp['sum_in_billions'] = round(temp['sum'] / 10**9,2)

#calculationg percentage distribution of purchase amount
temp['%sum'] = round(temp['sum']/temp['sum'].sum(),3)

#calculationg per purchase amount
temp['per_purchase'] = round(temp['sum']/temp['count'])
```


temp

Out[110...

	Age	sum	count	sum_in_billions	%sum	per_purchase
0	0-17	134913183	15102	0.13	0.026	8933.0
1	18-25	913848675	99660	0.91	0.179	9170.0
2	26-35	2031770578	219587	2.03	0.399	9253.0
3	36-45	1026569884	110013	1.03	0.201	9331.0
4	46-50	420843403	45701	0.42	0.083	9209.0
5	51-55	367099644	38501	0.37	0.072	9535.0
6	55+	200767375	21504	0.20	0.039	9336.0

In [112...

```
import matplotlib.pyplot as plt
import seaborn as sns

# 🍷 Better gradient palette
color_map = ["#DCEEF2", "#BBDCE5", "#8FC1D4", "#5FA8C6", "#3A8FB7", "#2E6F95", "#1B4F72"]

fig = plt.figure(figsize=(20, 14))
gs = fig.add_gridspec(3, 1, height_ratios=[0.12, 0.38, 0.5])

# ----- Stacked bar -----
ax0 = fig.add_subplot(gs[0])

left = 0

for i in temp.index:
    width = temp.loc[i, '%sum']

    ax0.barh([''], width, left=left, color=color_map[i])

    # text
    ax0.text(left + width/2, 0,
             f"{temp.loc[i, 'Age']}\n{temp.loc[i, 'sum_in_billions']}B",
             ha='center', va='center',
             fontsize=12, color='white', fontweight='bold')

    left += width

for s in ['top', 'left', 'right', 'bottom']:
    ax0.spines[s].set_visible(False)

ax0.set_xticks([])
ax0.set_yticks([])
ax0.set_xlim(0,1)

ax0.set_title('Age Group Purchase Contribution',
              fontsize=15, fontweight='bold', fontfamily='serif')

# ----- Avg purchase -----
ax1 = fig.add_subplot(gs[1])

bars = ax1.bar(temp['Age'], temp['per_purchase'],
               color=color_map, width=0.4)

avg = df['Purchase'].mean()
ax1.axhline(avg, color='#E63946', linestyle='--')

ax1.text(2, avg + 200,
         f"Overall Avg = ${avg:.0f}",
         fontsize=11)

for bar, val in zip(bars, temp['per_purchase']):
    ax1.text(bar.get_x()+bar.get_width()/2, val/2,
             f"${val:.0f}",
             ha='center', color='white', fontweight='bold')

ax1.grid(axis='y', alpha=0.3)

for s in ['top', 'right', 'left']:
    ax1.spines[s].set_visible(False)

ax1.set_ylabel('Purchase Amount', fontsize=12, fontweight='bold')

ax1.set_title('Avg Purchase per Transaction by Age',
              fontsize=15, fontweight='bold', fontfamily='serif')

# ----- Boxplot (REPLACES KDE) -----
ax2 = fig.add_subplot(gs[2])

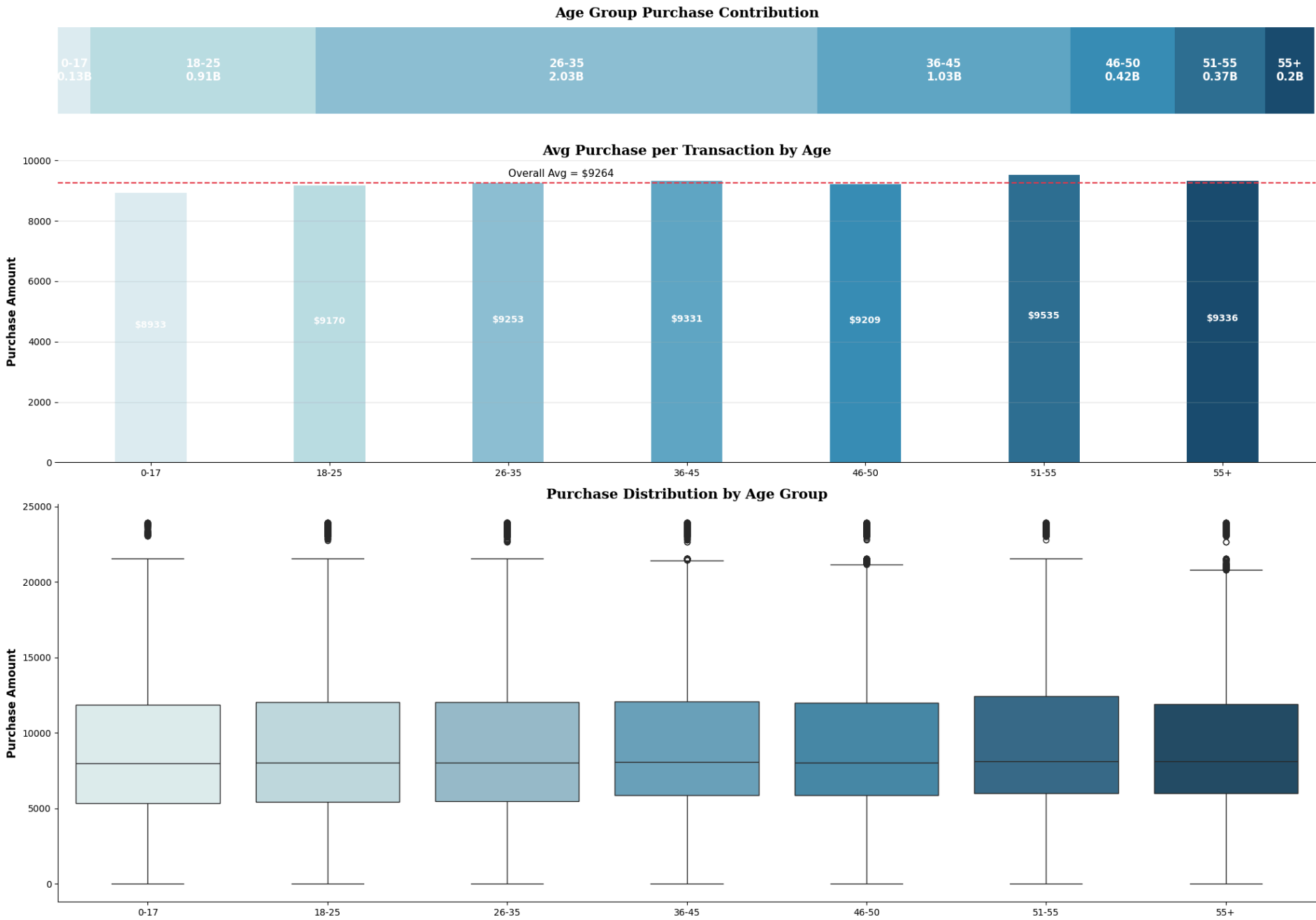
sns.boxplot(
    data=df,
    x='Age',
    y='Purchase',
    palette=color_map,
    ax=ax2
)
```

```
for s in ['top', 'right']:
    ax2.spines[s].set_visible(False)

ax2.set_xlabel('')
ax2.set_ylabel('Purchase Amount', fontsize=12, fontweight='bold')

ax2.set_title('Purchase Distribution by Age Group',
              fontsize=15, fontweight='bold', fontfamily='serif')

plt.tight_layout()
plt.show()
```



🔍 Insights

- **26–35 dominates revenue (~\$2.03B)**, clearly the core customer segment driving sales.
- **Average spend is almost flat across ages (~9.1k–9.5k)**, meaning no age group spends significantly more per transaction.
- **Older groups (51–55, 55+) show slightly higher averages**, but the difference is marginal and not practically significant.
- **Boxplots show very similar medians and spreads**, confirming that spending behavior is consistent across age groups.

Age impacts participation (who buys), but not spending behavior (how much they spend).

7.2 Confidence Interval Construction: Estimating Average Purchase Amount per Transaction

1. Step 1 - Building CLT Curve

- As seen above, the purchase amount distribution is not Normal. So we need to use Central Limit Theorem. It states the distribution of sample means will approximate a normal distribution, regardless of the underlying population distribution

2. Step 2 - Building Confidence Interval

- After building CLT curve, we will create a confidence interval predicting population mean at 95% Confidence level.

Note - We will use different sample sizes of [100,1000,5000,50000]

In [114...

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

def plot(ci):
    fig = plt.figure(figsize=(15, 15))
    gs = fig.add_gridspec(4, 1)

    age_groups = ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
    color_map = {
        '0-17': '#3A7089',
        '18-25': '#4b4b4c',
        '26-35': '#99AEBB',
        '36-45': '#5C8374',
    }
```

```

'46-50': '#6F7597',
'51-55': '#7A9D54',
'55+': '#9EB384'
}

# create series for each age group
age_data = {age: df.loc[df['Age'] == age, 'Purchase'] for age in age_groups}

sample_sizes = [(100, 0), (1000, 1), (5000, 2), (50000, 3)]
bootstrap_samples = 20000

# store results
all_samples = {age: {} for age in age_groups}

for size, row in sample_sizes:
    temp_dict = {age: [] for age in age_groups}

    for _ in range(bootstrap_samples):
        for age in age_groups:
            sample = np.random.choice(age_data[age], size=size)
            temp_dict[age].append(np.mean(sample))

# save in final dictionary
for age in age_groups:
    all_samples[age][f'{ci}%_{size}'] = temp_dict[age]

temp_df = pd.DataFrame(temp_dict)

ax = fig.add_subplot(gs[row])

for age in age_groups:
    sns.kdeplot(
        data=temp_df,
        x=age,
        color=color_map[age],
        fill=True,
        alpha=0.25,
        linewidth=1.8,
        ax=ax,
        label=age
    )

for s in ['top', 'left', 'right']:
    ax.spines[s].set_visible(False)

ax.set_yticks([])
ax.set_ylabel('')
ax.set_xlabel('')
ax.set_title(
    f'CLT Curve for Sample Size = {size}',
    fontdict={'family': 'serif', 'size': 11, 'weight': 'bold'}
)

fig.suptitle(f'{ci}% Confidence Interval', font='serif', size=18, weight='bold')
handles, labels = ax.get_legend_handles_labels()
fig.legend(handles, labels, loc='center left', bbox_to_anchor=(1.02, 0.5), frameon=False)

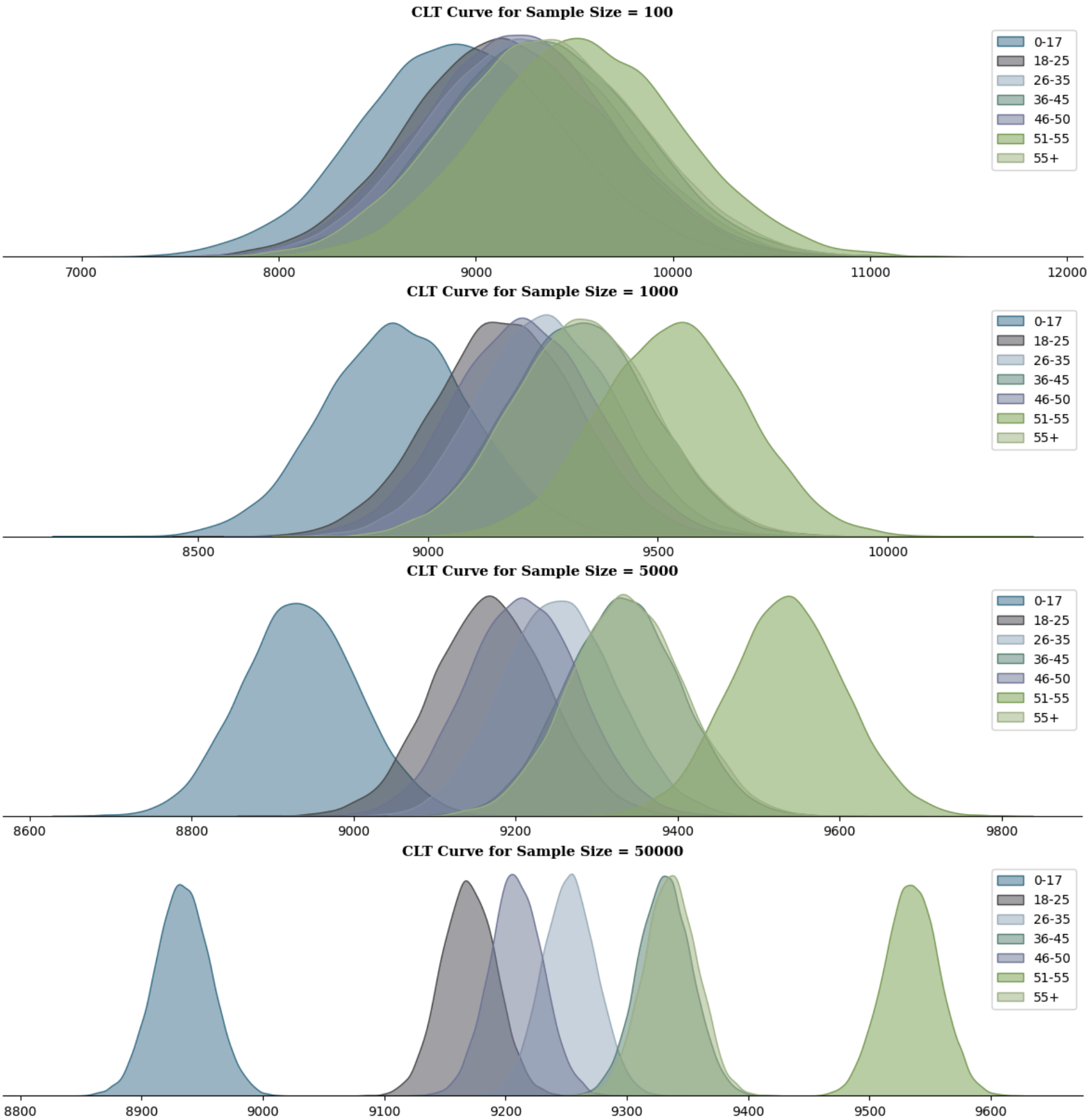
plt.tight_layout(rect=[0, 0, 0.88, 0.97])
plt.show()

return all_samples

samples_age_95 = plot(95)

```

95% Confidence Interval



Are confidence intervals of customer's age-group spending overlapping?

In [116...

```
print("\n95% Confidence Interval Summary (Age Groups)")
print("-" * 60)

age_labels = ['0-17', '18-25', '26-35', '36-45', '46-50', '51-55', '55+']
samples = [samples1, samples2, samples3, samples4, samples5, samples6, samples7]

# sort keys by sample size
keys = sorted(samples1.keys(), key=lambda x: int(x.split('_')[1]))

for key in keys:
    size = key.split('_')[1]

    print(f"\nSample Size = {size}")
    print("-" * 30)

    for age, sample_dict in zip(age_labels, samples):
        low, high = confidence_interval(sample_dict[key], 95)
        rng = high - low

        print(f"{age:<6} : ${low:.0f} - ${high:.0f} (Range = {rng:.0f})")
```

Sample Size = 100

0-17	:	\$7934	-	\$9952	(Range = 2018)
18-25	:	\$8213	-	\$10168	(Range = 1955)
26-35	:	\$8283	-	\$10248	(Range = 1965)
36-45	:	\$8358	-	\$10318	(Range = 1960)
46-50	:	\$8260	-	\$10201	(Range = 1941)
51-55	:	\$8570	-	\$10548	(Range = 1978)
55+	:	\$8372	-	\$10340	(Range = 1968)

Sample Size = 1000

0-17	:	\$8615	-	\$9256	(Range = 641)
18-25	:	\$8856	-	\$9481	(Range = 625)
26-35	:	\$8947	-	\$9559	(Range = 612)
36-45	:	\$9023	-	\$9641	(Range = 618)
46-50	:	\$8902	-	\$9519	(Range = 617)
51-55	:	\$9217	-	\$9850	(Range = 633)
55+	:	\$9022	-	\$9649	(Range = 627)

Sample Size = 5000

0-17	:	\$8795	-	\$9076	(Range = 281)
18-25	:	\$9029	-	\$9310	(Range = 281)
26-35	:	\$9116	-	\$9393	(Range = 277)
36-45	:	\$9193	-	\$9471	(Range = 278)
46-50	:	\$9072	-	\$9346	(Range = 274)
51-55	:	\$9395	-	\$9677	(Range = 282)
55+	:	\$9198	-	\$9476	(Range = 278)

Sample Size = 50000

0-17	:	\$8889	-	\$8979	(Range = 90)
18-25	:	\$9126	-	\$9214	(Range = 88)
26-35	:	\$9208	-	\$9297	(Range = 89)
36-45	:	\$9287	-	\$9376	(Range = 89)
46-50	:	\$9165	-	\$9253	(Range = 88)
51-55	:	\$9490	-	\$9579	(Range = 89)
55+	:	\$9293	-	\$9379	(Range = 86)

Here’s a cleaner, sharper version of your insights (same meaning, tighter + more professional):

Insights

1. Sample Size Effect

As sample size increases, confidence intervals become **narrower and more precise**, improving the reliability of estimates. This reinforces that **larger samples lead to better business decisions**.

2. Spending Patterns by Age

- **0–17**: Lowest average spending per transaction.
- **18–25, 26–35, 46–50**: Overlapping confidence intervals → **similar spending behavior**.
- **36–45, 55+**: Also show overlapping intervals → **comparable patterns**.
- **51–55**: Highest average spending → **top-value segment**.

3. Population Averages (95% CI)

We are 95% confident that the true average purchase lies within:

- 0–17: 8, 888–8,979
- 18–25: 9, 125–9,213
- 26–35: 9, 209–9,297
- 36–45: 9, 288–9,376
- 46–50: 9, 165–9,253
- 51–55: 9, 490–9,579
- 55+: 9, 292–9,381

Business Recommendations

4.1 Targeted Marketing

Increase spending in **0–17** using discounts, bundles, and youth-focused campaigns.

4.2 Customer Segmentation

Group similar segments (**18–25, 26–35, 46–50** and **36–45, 55+**) to **optimize pricing and product strategies efficiently**.

4.3 Premium Strategy

Leverage **51–55 high spenders** with premium offerings, personalized deals, and loyalty programs.

Age influences spending slightly, but most groups behave similarly, with only a few high- and low-value segments standing out.

8 Recommendations

- Prioritize Male Segment** Lean into male-heavy demand with targeted promos and bundles, while maintaining strong value propositions for female shoppers to balance volume.
- Double Down on 26–45** This core segment drives most revenue—push **category-specific deals, bundles, and timed offers** tailored to their preferences.
- Activate Younger Shoppers (0–17)** Boost basket size via **entry-level bundles, coupons, and gamified rewards** to start early loyalty.
- Smart Segmentation** Treat **18–25, 26–35, 46–50** as one cluster and **36–45, 55+** as another to **streamline pricing, merchandising, and campaigns**.
- Premium Play for 51–55** Target top spenders with **early access, premium bundles, and personalized recommendations** to maximize high-value conversions.
- Post-Event Retention** Use Black Friday data for **follow-up offers, cross-sell emails, and remarketing** to drive repeat purchases.

Focus on high-volume segments for scale, high-spending segments for margin, and use segmentation + retention to sustain growth.

In []: